Module-1:Core Java

Introduction

- Java programming language was originally developed by Sun Microsystems.
- It is initiated by James Gosling and released in 1995 as core components of sun microsystems' java platform (java 1.0 [J2SE].
- Sun Microsystems was acquired by the oracle corporation.
- Now it is part of Oracle.
- Java is guaranteed to be write once, run anywhere.
- Platforms Available:
  - ❑J2SE for Standard Edition
  - ❑J2EE for Enterprise Applications
  - ❑ J2ME for Mobile Applications

Feature of Java:

- Java is simple
- Java is object-oriented
- Java is distributed
- Java is interpreted
- Java is robust/powerful
- Java is secure
- Java is architecture-neutral
- Java portable
- Java performance
- Java is multithreaded
- Java is dynamic.

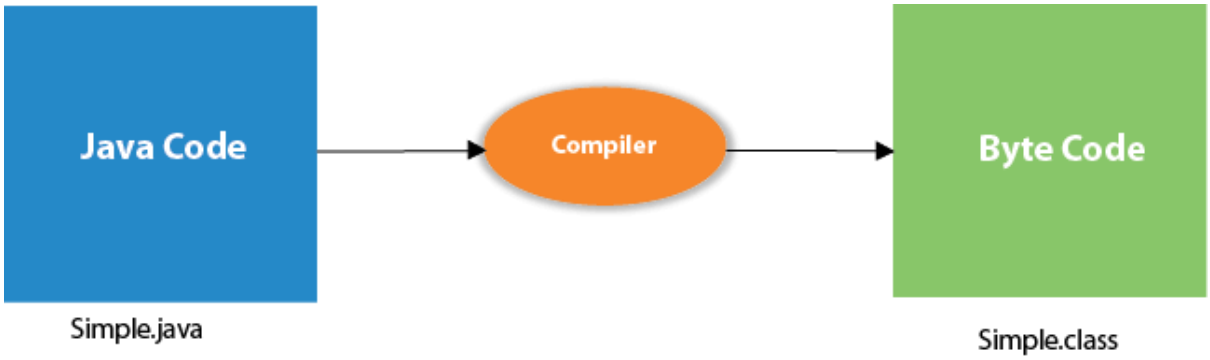
Advantages of Java:

- It is an open source, so users do not have to struggle with heavy licence fees each year.
- Platform independent.
- Java API's can easily be accessed by the developers.
- Java perform supports garbage collection, so memory management is automatic.
- Java always allocates object on the heap.
- Java embraced the concepts of exception specifications.
- Multiplatform support language and support for web services.
- Using java we can develop web applications.
- It allows you to create modular programs and reusable codes.

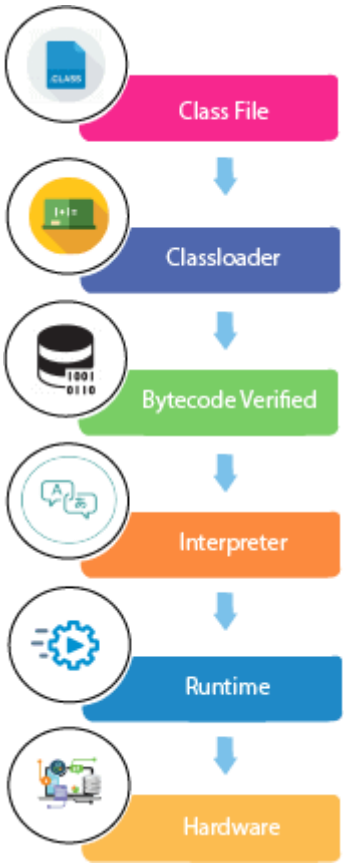
Introduction of JVM JRE and JDK:

- JVM stands for Java Virtual Machine.
- It is the engine that drives the java code.
- It is the medium which compile java code to bytecode which get interpret on different machine and hence it makes it Platform/Operating system independent. source and the host
- Bytecode is an intermediary language between Java system.
- Java is called platform independent because of Java Virtual Machine.

- As different computers with different operating system have their own JVM, when we submit '.class' file to any operating system, JVM interpret the bytecode into machine level language



What happens at runtime?
At runtime, the following steps are performed:



JDK(Java Development Kit)

- Java Development Kit contains the software and tools that you need to compile, debug, and run applets and applications that you've written using the Java programminglanguage.
- The JDK has as its primary components a collection of programming tools . This toolalso helps manage JAR files, javadoc .

- The JDK also comes with a complete Java Runtime Environment, usually called a private runtime. It consists of a Java Virtual Machine and all of the class libraries present in the production environment, as well as additional libraries only useful to developers.

Java Versions:

- JDK Alpha and Beta(1995)
- JDK1.0(January23,1996)
- JDK1.1(February19,1997)
- J2SE1.2(December8,1998)
- J2SE1.3(May8,2000)
- J2SE1.4(February6,2002)
- J2SE5.0(September30,2004)
- JavaSE6(December11,2006)
- JavaSE7(July 28,2011)
- JavaSE8(March18,2014).

JRE:

- The Java Runtime Environment as Java minimum (JRE), known also Runtime, is part of the Java Development Kit (JDK),
- A set of programming tools for developing Java applications.
- The Java Runtime Environment provides requirements for executing a Java application.

Java Virtual Machin(JVM):

- It consists the (JVM), core and of the Java Virtual Machine components, which are necessary to run programs written in Java. The JRE is available for multiple computer platforms, including Mac, Windows, and Unix

Java IDE Tools:

- To write your Java programs, you will need a text editor. There are more sophisticated IDEs available in the market.
- Notepad: On Windows machine you can use any simple text editor like Notepad, Text Pad.
- Net beans: is a Java IDE that is open-source. ❑
- Eclipse: is also a Java IDE developed by the eclipse open-source community.

About Eclipse IDE :

- Most development people knowEclipse as an integrated environment (IDE)for Java.
- Today it is the leading development environment for Java with a market share of approximately 65%.
- The Eclipse IDE can software
- Eclipse Several components. calls these Open be extended software components with additional plug-ins. Source projects and companies have extended the Eclipse IDE.

Introduction:

Class Object and Members:

Class : A class can be define as a template/blue print that describes behaviour/state that object of its type support.

1. Objects are key to understanding object-oriented technology.
2. Examples of real-world objects:

    - your dog,
    - your desk,
    - your television set,
    - your bicycle.

3. Real-world objects share two characteristics:
4. They all have state and behaviour.

Example:

A dog has states- colour, name, breed as well as behaviours barking, eating.

An object is an instance of a class. wagging,

5. An object is an instance of a class created using a new operator.
6. The new operator returns a reference toa new instance of a class.
7. Eg Person me=new Person(); Class Members
8. When we create a class its totally incomplete without defining any member of this class.

Field: Field is nothing but the property of the class or object which we are going to create for example if we are creating a class called computer then they have property like model, mem_size, hd_size, os_type etc.

Method: method is nothing but the operation that an object can perform it define the behaviour of object how an object can interact without side world .

For example : startMethod (), shutdownMethod().

Simple problem :

```
public class HelloWorld {
public static void main(String[] args){
// Print Hello World to the console
System.out.println("Hello, World!");
} }
```

Variable:

What is a Variable?

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.

Types of Java Variables

There are three types of variables in Java:

- Java Local Variable
- Java Instance Variable
- Java Static Variable

Java Local Variable:

A variable declared inside the body of the method is called local variable.

```
//defining a Local Variable
int num = 10;
System.out.println(" Variable: " + num);
```

Java Instance Variable:

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

```
public class InstanceVariableDemo {
   //Defining Instance Variables
   public String name;
   public int age=19;
 //Creadting a default Constructor initializing Instance Variable
   public InstanceVariableDemo()
   {
      this.name = "Deepak";
   }
}
public class Main{
   public static void main(String[] args)
   {
      // Object Creation
     InstanceVariableDemo obj = new InstanceVariableDemo();
      System.out.println("Student Name is: " + obj.name);
      System.out.println("Age: "+ obj.age);
   }
}
```

Java Static Variable:

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

```
class Student{
  //static variable
  static int age;
}
public class Main{
  public static void main(String args[]){
    Student s1 = new Student();
    Student s2 = new Student();
    s1.age = 24;
    s2.age = 21;
    Student.age = 23;
    System.out.println("S1\'s age is: " + s1.age);
    System.out.println("S2\'s age is: " + s2.age);
  }
}
```

Data Types:

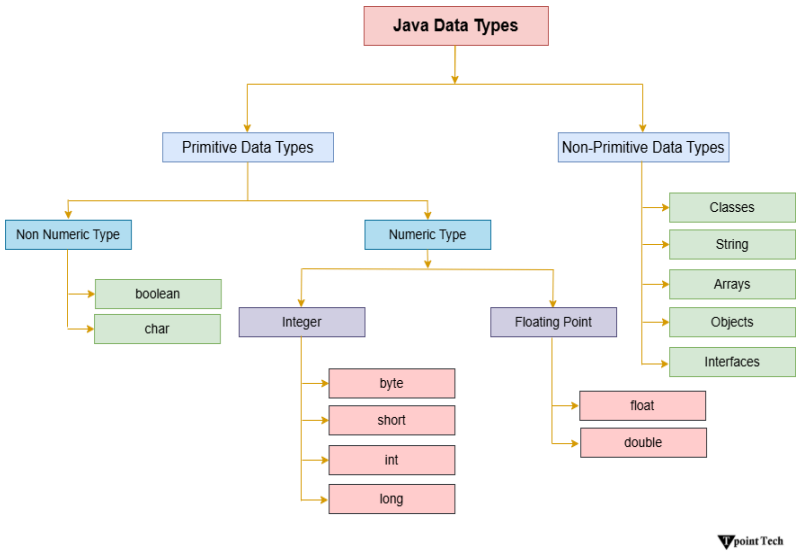
Datatypes are nothing but reserved memory locations

Variables values.

- Based on the data type of a variable, the operating allocates memory and decides what can be stored reserved memory.

There are two data types available in Java

- Primitive DataTypes
- Reference/Object Data Type

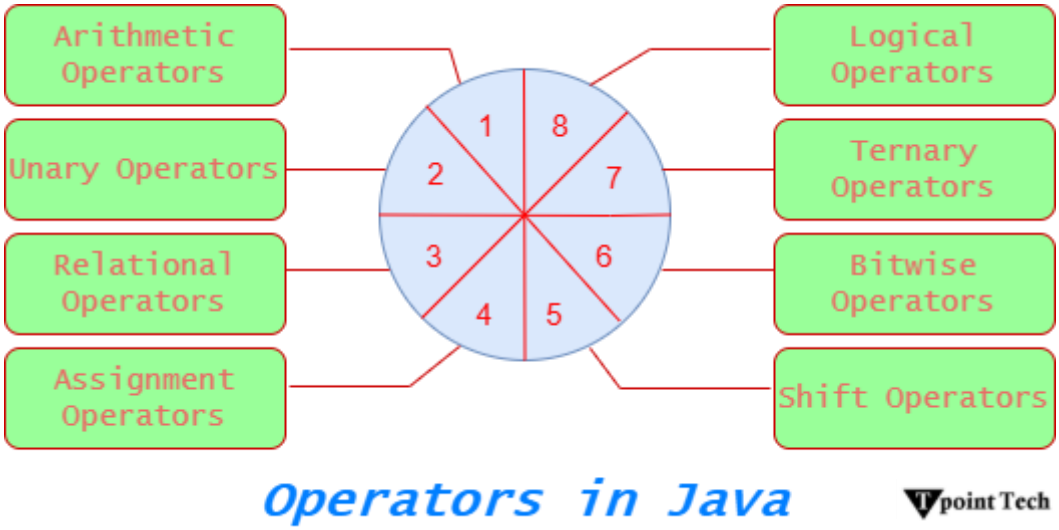


Java Operators

Operators are an essential part of any programming language. In Java, operator is a symbol that is used to perform operations. For example: +, -, *, / etc. These are essential for performing different types of operations on variables and values. In this section, we will discuss **different types of operators used in Java programming**.

There are mainly eight types of operators in Java:

1. Unary Operator
2. Arithmetic Operator



3. Relational Operator
4. Ternary Operator
5. Assignment Operator
6. Bitwise Operator
7. Logical Operator
8. Shift Operator

TypeCasting:

Assigning a value of one type to a variable of a no their type is known as TypeCasting. Widening or Automatic type conversion Automatic Type casting take place when, the two types are Compatible the target type is larger than the source type.

Conditional Statement and Looping Statement :

❑If, ❑If ,else if, ❑Switch..case ❑For ❑While ❑Do…while

There are two types of decision making statements in Java.Theyare:

❑If statement ❑Switch statement

The if Statement:

An if statement consists of a Boolean expression followed by one or more statements.

Syntax: if(Boolean_expression) {
//Statements will execute if the Boolean expression is true
}
If the Boolean expression evaluates to true then the block of code inside the if statement will be executed. If not the first set of code after the end of the if statement(after the closing curly brace) will be executed.

The if...else Statement:

An if statement can be followed by an optional else statement, which executes when the Boolean expression is false.

Syntax: if(Boolean_expression){
//Executes when the Boolean expression is true
}
else{
//Executes when the Boolean expression is false

Theif...else if...else Statement:
An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.
Syntax: if(Boolean_expression 1){
//Executes when the Booleanexpression 1 is true
}else if(Boolean_expression 2){
else if(Boolean_expression)
//Executes when the Boolean expression 2 is true
}
3){
//Executes when the Boolean expression 3 is true
} else {
//Executes when the none of the above condition is true
}

The switch Statement:
A switch statement allows a variable to be tested for equality against alist of values.
Each value is called a case, and the variable being switched on is checked for each case.
Syntax: switch(expression){
case value://Statementsbreak;
//optional
case value: //Statements break;
//optional
//You can have any number of case statements.
default : //Optional //Statements

Looping Statement:

Loops are specifically designed to perform repetitive tasks with one set of code. Loops save a lot of time. There are:

while loop
do while loop &
for loop

The while loop:

Syntax:

while ( )
<statement_or_block>

The do while loop:

Syntax:
do{
<statement_or_block>
}while (<test-expr>);

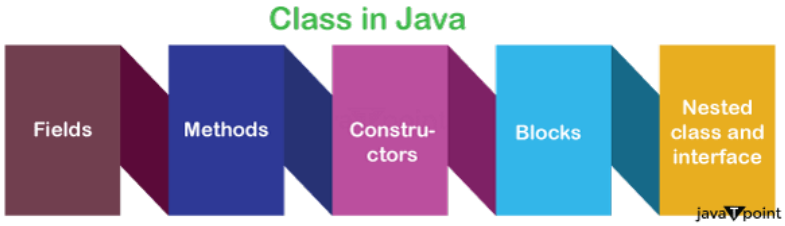
The for loop:

Syntax:
for(initialization; condition; increment/decrement){
//statement or code to be executed
}

Java Break Statement:
The `break` statement in Java is a powerful tool for controlling the flow of your program. It is used to exit a loop or switch statement prematurely. It allows us to customize the behavior of code based on specific conditions.
Syntax of the break Statement:
jump-statement;
break;
continue Statement in Java:
The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.
Syntax of continue Statement
It has the following syntax:
jump-statement;
continue;

Java Classes and Objects

A class is a group of objects that have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.
A Java class contains:



Java Class Syntax:

class <class_name>{
field;
method;
}

Object:

An object is a real-world entity that has state and behaviour.
Syntax of an Object:
<class_name> variable=**new** <class_name>();

Initializing an Object in Java
There are the following three ways to initialize an object in Java:

1. By Reference Variable
2. By Method
3. By Constructor

Constructors in Java:

In Java, a constructor is a special method used to initialize objects when a class is created.

Syntax:

It has the following syntax:

```
class Student {
   Student() {
      System.out.println("Constructor called");
   }
}
```

Types of Java Constructors

There are two types of constructors in Java:

1. **Default Constructor (No-arg Constructor):**
   The default constructor does not take any parameters and is used to initialize an object with default values. If no constructor is defined, Java automatically provides a default constructor.

2. **Parameterized Constructor:**
   The parameterized constructor accepts parameters and is used to initialize an object with specific values at the time of object creation.

3. **Copy Constructor:**
   A copy constructor is a special type of constructor that is commonly utilized to create a new object by copying the values of an existing object of the same class.

1. Java Default Constructor:
When a constructor does not have any parameter, is known as default constructor.
Syntax
It has the following syntax:
<class_name>(){}
2. Java Parameterized Constructor
A constructor that has a specific number of parameters is called a parameterized constructor.
Syntax

```
1. class ClassName {
2.     ClassName(dataType parameter1, dataType parameter2, ...) {
3.         // constructor body
4.     }
```

5. }

Java Copy Constructor

Java does not support the copy constructor. However, we can copy the values from one object to another, like a copy constructor in C++.

There are the following three ways to copy the values of one object into another:

- By Using a Constructor
- By Assigning the Values of One Object to Another
- By Using the clone() Method of the Object Class

Method Overloading in Java:

Method Overloading in Java allows us to create multiple methods with the same name to perform similar tasks using different parameters.

)

Object Initialization through Reference Variable

Initializing an object means storing data in the object. Let's see a simple example where we are going to initialize the object through a reference variable.

Example

```
class Student{
 int id;
 String name;
}
public class Main{
 public static void main(String args[]){
  //Creating instance of Student class
  Student s1=new Student();
  //assigning values through reference variable
  s1.id=101;
  s1.name="Sonoo";
  //printing values of s1 object
  System.out.println(s1.id+" "+s1.name);
 }
}
```

Output:

101 Sonoo

Example:

```
class Student{
 int id;
 String name;
}
public class Main{
 public static void main(String args[]){
  //Creating objects
  Student s1=new Student();
  Student s2=new Student();
  //Initializing objects
  s1.id=101;
  s1.name="Sonoo";
  s2.id=102;
```

```
 s2.name="Amit";
 //Printing data
 System.out.println(s1.id+" "+s1.name);
 System.out.println(s2.id+" "+s2.name);
 }
}
```

Output:

101 Sonoo
102 Amit

2) Object Initialization through Method

In this example, we are creating the two objects of Student class and initializing the value to these objects by invoking the insertRecord method.

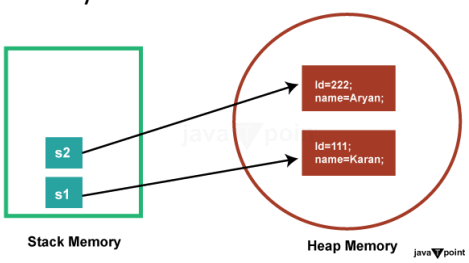
Here, we are displaying the state (data) of the objects by invoking the displayInformation() method.

Example

```
class Student{
 int rollno;
 String name;
 //Creating a method to insert record
 void insertRecord(int r, String n){
  rollno=r;
  name=n;
 }
 //creating a method to display information
 void displayInformation(){System.out.println(rollno+" "+name);}
}
//Creating a Main class to create objects and call methods
public class Main{
 public static void main(String args[]){
  Student s1=new Student();
  Student s2=new Student();
  s1.insertRecord(111,"Karan");
  s2.insertRecord(222,"Aryan");
  s1.displayInformation();
  s2.displayInformation();
 }
}
```

Output:

111 Karan
222 Aryan



3) Object Initialization through a Constructor

The concept of object initialization through a constructor is essential to object-oriented programming in Java. Special methods inside a class called constructors are called when an object of that class is created with the new keyword. They initialise the state of objects by entering initial values in their fields or carrying out any required setup procedures. The constructor is automatically invoked upon object instantiation, guaranteeing correct initialization of the object prior to usage.

If you are a beginner, skip this part because we will learn about the constructor later.

Here's an example demonstrating object initialization through a constructor.

Example

```
class Student {
   int id;
   String name;
   // Constructor with parameters
   public Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
   // Method to display student information
   public void displayInformation() {
      System.out.println("Student ID: " + id);
      System.out.println("Student Name: " + name);
   }
}
public class Main {
   public static void main(String[] args) {
      // Creating objects of Student class with constructor
      Student student1 = new Student(1, "John Doe");
      Student student2 = new Student(2, "Jane Smith");
      // Displaying information of the objects
      student1.displayInformation();
      student2.displayInformation();
      1.       }

      2.   }
```

Output:

Student ID: 1

Student Name: John Doe

Student ID: 2

Student Name: Jane Smith

Let's see an example where we are maintaining records of employees.

Example

```
class Employee{
   int id;
   String name;
   float salary;
```

```
   void insert(int i, String n, float s) {
      id=i;
      name=n;
      salary=s;
   }
   void display(){System.out.println(id+" "+name+" "+salary);}
}
public class Main {
public static void main(String[] args) {
   Employee e1=new Employee();
   Employee e2=new Employee();
   Employee e3=new Employee();
   e1.insert(101,"ajeet",45000);
   e2.insert(102,"irfan",25000);
   e3.insert(103,"nakul",55000);
   e1.display();
   e2.display();
   e3.display();
}
}
```

Output:

101 ajeet 45000.0
102 irfan 25000.0
103 nakul 55000.0

Explanation

In the above program, the Employee class has three fields: id, name, and salary. It also has two methods, insert(), which sets the values for these fields, and display(), which prints the values. The main() function of the Main class creates three Employee objects (e1, e2, and e3). It initialises the id, name, and salary fields of each object with precise values, and the insert() method is called on each of the objects. Then, each object's display() method is called, displaying object initialization and information display via method invocation. Each object's id, name, and salary values are printed to the console.

Object and Class Example: Rectangle

Another example is given that maintains the records of the Rectangle class.

Example

```
class Rectangle{
 int length;
 int width;
 void insert(int l, int w){
  length=l;
  width=w;
 }
 void calculateArea(){System.out.println(length*width);}
}
public class Main{
 public static void main(String args[]){
  Rectangle r1=new Rectangle();
  Rectangle r2=new Rectangle();
```

```
  r1.insert(11,5);
  r2.insert(3,15);
  r1.calculateArea();
  r2.calculateArea();
}
}
```

Output:

55
45

Explanation

This Java code defines a Rectangle class with fields for length and width, along with methods to insert dimensions and calculate the area. In the Main class's main() method, two Rectangle objects are instantiated, and their dimensions are set using the insert method.

What are the different ways to create an object in Java?

There are the following ways to create an object in Java.

1. By new Keyword

The most common way to create an object in Java is using the new keyword followed by a constructor. For example,

1. ClassName obj = **new** ClassName();

It allocates memory for the object and calls its constructor to initialize it.

2. By newInstance() Method

This method is part of the java.lang.Class class and is used to create a new instance of a class dynamically at runtime. It invokes the no-argument constructor of the class. For example,

1. ClassName obj = (ClassName) Class.forName("ClassName").newInstance();.

3. By clone() Method

The clone() method creates a copy of an existing object by performing a shallow copy. It returns a new object that is a duplicate of the original object. For example,

1. ClassName obj2 = (ClassName) obj1.clone();.

4. By Deserialization

Objects can be created by deserializing them from a stream of bytes. We can achieve it by using the ObjectInputStream class in Java. The serialized object is read from a file or network, and then the readObject() method is called to recreate the object.

5. By factory Method

Factory methods are static methods within a class that return instances of the class. They provide a way to create objects without directly invoking a constructor and can be used to encapsulate object creation logic. For example,

1. ClassName obj = ClassName.createInstance().

Anonymous object:

Anonymous simply means nameless. An object that has no reference is known as an anonymous object. It can be used at the time of object creation only.

If we have to use an object only once, an anonymous object is a good approach. For example:

1. **new** Calculation();*//anonymous object*

Calling a method through an anonymous object

1. Calculation c=**new** Calculation();
2. c.fact(5);

Calling method through an anonymous object

1. **new** Calculation().fact(5);

Let's see the full example of an anonymous object in Java.

```
Example
class Main{
void fact(int  n){
int fact=1;
for(int i=1;i<=n;i++){
fact=fact*i;
}
System.out.println("factorial is "+fact);
}

public static void main(String args[]){
new Main().fact(5);//calling method with anonymous object
}
}
```

Output:

Factorial is 120

Initialization of Reference Variables

1. Rectangle r1=**new** Rectangle(), r2=**new** Rectangle();*//creating two objects*

Let's see an example.

Example

```
//Java Program to illustrate the use of the Rectangle class, which
//has length and width data members
class Rectangle{
int length;
int width;
void insert(int l,int w){
length=l;
width=w;
}
void calculateArea(){System.out.println(length*width);}
}
```

```
class Main{
public static void main(String args[]){
Rectangle r1=new Rectangle(),r2=new Rectangle();//creating two objects
r1.insert(11,5);
r2.insert(3,15);
r1.calculateArea();
r2.calculateArea();
}
}
```

Output:

55

45

Methods in Java

In Java, a method is used to do a specific work and helps make the program simple and reusable.

Advantages of Using Methods

The following are the advantages of using methods in Java:

- ***Reusability:** Methods provide code reusability. Once a method is defined, it can be called multiple times from various parts of the program.*
- ***Modularity:** Methods help break down complex problems into smaller, manageable pieces.*
- ***Maintainability:** With methods, updating and debugging code becomes easier since changes in one part of the program do not necessarily impact other parts.*
- ***Abstraction:** Methods provide a way to encapsulate implementation details, exposing only the necessary parts to the users.*

Method Naming Rules

While defining a method, remember that the method name must be a verb and start with a lowercase letter. If the method name has more than two words, the first name must be a verb followed by an adjective or noun. In the multi-word method name, the first letter of each word must be in uppercase except the first word. For example:

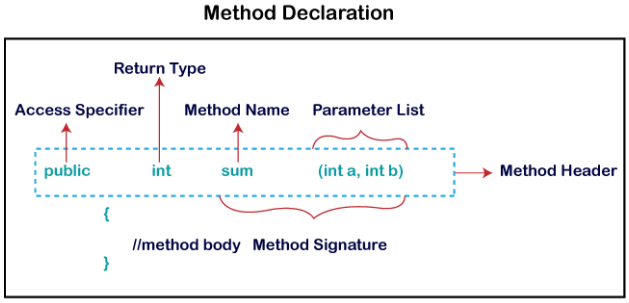
***Single-Word Method Name:** sum(), area()*

***Multi-Word Method Name:** areaOfCircle(), stringComparision()*

It is also possible that a method has the same name as another method in the same class; it is known as method overloading.

Method Declaration

The method declaration provides information about method attributes, such as visibility, return type, name, and arguments. It has six components that are known as method header, as we have shown in the following figure.



Method Signature

Every method has a method signature. It is a part of the method declaration. It includes the method name and parameter list.

Access Specifier

Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifiers:

- ***public:*** *The method is accessible to all classes when we use the public specifier in our application.*
- ***private:*** *When we use a private access specifier, the method is accessible only in the classes in which it is defined.*
- ***protected:*** *When we use a protected access specifier, the method is accessible within the same package or subclasses in a different package.*
- ***default:*** *When we do not use any access specifier in the method declaration, Java uses the default access specifier by default. It is visible only from the same package.*

Return Type

Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use the void keyword.

Method Name

It is a unique name that is used to define the name of a method. It must correspond to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be subtraction(). A method is invoked by its name.
Parameter List
It is the list of parameters separated by a comma and enclosed in a pair of parentheses. It contains the data type and variable name. If the method has no parameter, leave the parentheses blank.
Method Body
It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within a pair of curly braces.
The method body is enclosed in curly braces {} and contains the statements that define the functionality or working of the method. For example, consider the following code snippet.

```
public class Example {
   // Method definition
   public int add(int a, int b) {
      int sum = a + b; // Method body
      return sum; // Return statement
   }
}
```

Types of Methods
There are two types of methods in Java:

- *Predefined Method*
- *User-defined Method*

Types of Methods in Java

1. Predefined Method
The methods that are already defined in the Java class libraries are known as ***predefined methods****. It is also known as the* ***standard library method*** *or* ***built-in method****. We can directly use these methods just by calling them in the program at any point.*
For example, ***String.length(), String.equals(), String.compareTo(), Math.sqrt(), Math.pow(),*** *etc, are predefined methods.*
When we call any of the predefined methods in our program, a series of code related to the corresponding method runs in the background.

Example

```
public class Main {
  public static void main(String[] args) {
    // pow() is an in-built function defined in the Math class
    System.out.println("2 raised to the power of 5 is: " + Math.pow(2, 5));
  }
}
```

Output:

2 raised to the power of 5 is: 32.0

2. User-Defined Method

*The method written by the user or programmer is known as **a user-defined** method. These methods can be modified or customized according to the requirements.*

Calling User defined Method

In Java, calling or invoking a method depends on whether the method is static or non-static.

If the method is static, we can call it directly using the class name or from within the same class. Note that we need to create an object for calling static methods.

Syntax

It has the following syntax:

```
static void display() {
// code
}
Empolyee.display(); // no need to create an object
```

If the method is non-static, we must create an object of the class and use it to call the method.

***Note:** We will get a compile-time error if we try to call a non-static method from a static context without an object because non-static methods belong to instances, not the class itself.*

1. *Empolyee emp = **new** Empolyee();*
2. *emp.display(); // method called using object reference*

Example

Let us take an example to demonstrate how we can call the user defined function in Java.

```
public class Main {
  // Static method
  static void greet() {
    System.out.println("Hello from the static method!");
  }
  // Non-static method
  void farewell() {
    System.out.println("Goodbye from a non-static method!");
  }
  public static void main(String args[]) {
    Main obj = new Main();
    obj.farewell(); //calling non-static method
    Main.greet(); //calling static method
  }
}
```

Output:

Goodbye from a non-static method!

Hello from the static method!

Inheritance in Java

Last Updated : 10 Feb 2026

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that we can create new classes that are built upon existing classes. When we inherit methods from an existing class, we can reuse methods and fields of the parent class. However, we can add new methods and fields in your current class also.

What is Inheritance?

Inheritance in Java enables a class to inherit properties and actions from another class, called a superclass or parent class. A class derived from a superclass is called a subclass or child group. Through inheritance, a subclass can access members of its superclass (fields and methods), enforce reuse rules, and encourage hierarchy.

Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Why Use Inheritance in Java?

The inheritance is used for the following important reasons:

- ***Method Overriding:*** *Inheritance allows a subclass to provide a specific implementation of a method already defined in its superclass. This is also known as runtime polymorphism.*
- ***Code Reusability:*** *Inheritance allows developers to reuse the existing code from a parent class which reduces the redundancy and makes the code easier to maintain.*

Terms used in Inheritance

Here are the important terms (terminologies) that are used in inheritance:

- ***Class:*** *A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.*
- ***Sub Class/Child Class:*** *Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.*
- ***Super Class/Parent Class:*** *Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.*
- ***Reusability:*** *As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.*

The extends Keyword

*The **extends** keyword is used to create a new class (subclass) that derives from an existing class (superclass). It allows the subclass to inherit fields and methods from the superclass.*

Syntax

It has the following syntax:

```
1. class Subclass-name extends Superclass-name
2. {
3.    //methods and fields
4. }
```

Java Inheritance Example

*As displayed in the above figure, Programmer is the subclass and Employee is the superclass. The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.*

This example demonstrate the single inheritance in Java, where the Programmer class inherits the salary variable from the Employee class and also defines its own bonus.

Example

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
}
public class Main{
 public static void main(String args[]){
```

```
    Programmer p=new Programmer();
    System.out.println("Programmer salary is:"+p.salary);
    System.out.println("Bonus of Programmer is:"+p.bonus);
  }
  }
```

Output:

Programmer salary is:40000.0
Bonus of programmer is:10000

In the above example, Programmer object can access the field of own class as well as of Employee class i.e. code reusability.

Types of Inheritance

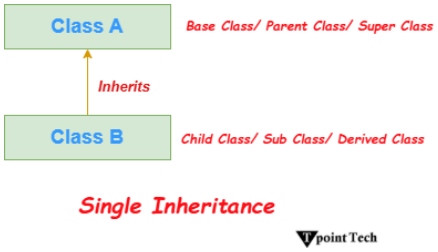
There are several types of inheritance supported in Java:

1. ***Single Inheritance:***
   Single inheritance allows a subclass inherits from one superclass only.
2. ***Multilevel Inheritance:***
   Multilevel inheritance allows a class is derived from a subclass, forming a chain of inheritance.
3. ***Hierarchical Inheritance:***
   Hierarchical inheritance allows multiple subclasses inherit from the same superclass.
4. ***Hybrid Inheritance (Through Interfaces):***
   Hybrid inheritance combines two or more types of inheritance using interfaces to avoid multiple inheritance issues.

In Java, multiple inheritance and hybrid inheritance are supported only through interfaces, because Java does not allow multiple inheritance with classes directly. We will learn more about interfaces later.

Single Inheritance

When a class inherits another class, it is known as a single inheritance. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.



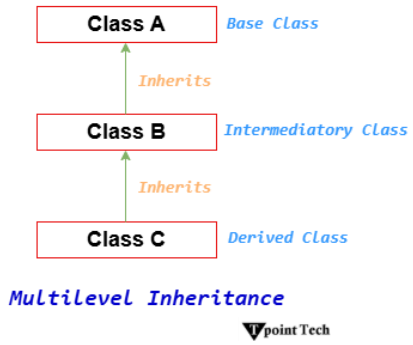
example to demonstrate the single inheritance in Java.

Example

```
  class Animal{
  void eat(){System.out.println("eating...");}
  }
  class Dog extends Animal{
  void bark(){System.out.println("barking...");}
  }
  public class Main{
  public static void main(String args[]){
  Dog d=new Dog();
  d.bark();
  d.eat();
1.  }}
```

Output:

barking...
eating...
Multilevel Inheritance
When there is a chain of inheritance, it is known as multilevel inheritance. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

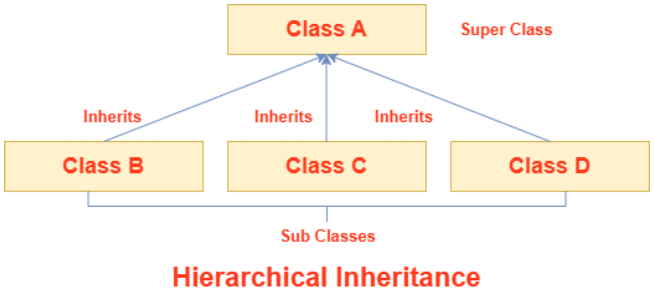


Here, we are going to take an example to demonstrate the working of multilevel inheritance in Java.
Example

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
public class Main{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:
weeping...
barking...
eating...

Hierarchical Inheritance
When two or more classes inherits a single class, it is known as hierarchical inheritance. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.



Hierarchical Inheritance

Here, we are going to take an example to demonstrate the working of hierarchical inheritance in Java.

Example

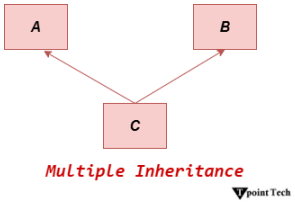
```
1.  class Animal{
2.  void eat(){System.out.println("eating...");}
3.  }
4.  class Dog extends Animal{
5.  void bark(){System.out.println("barking...");}
6.  }
7.  class Cat extends Animal{
8.  void meow(){System.out.println("meowing...");}
9.  }
10. public class Main{
11. public static void main(String args[]){
12. Cat c=new Cat();
13. c.meow();
14. c.eat();
15. //c.bark();//C.T.Error
16. }}
```

Output:

meowing...
eating...

Multiple Inheritance

Multiple Inheritance A class's capacity to inherit traits from several classes is referred to as multiple inheritances. This notion may be quite helpful when a class needs features from many sources.



example to demonstrate the working of multiple inheritance in Java.

Example

```
interface Character {
   void attack();
}
interface Weapon {
   void use();
}
class Warrior implements Character, Weapon {
   public void attack() {
      System.out.println("Warrior attacks with a sword.");
```

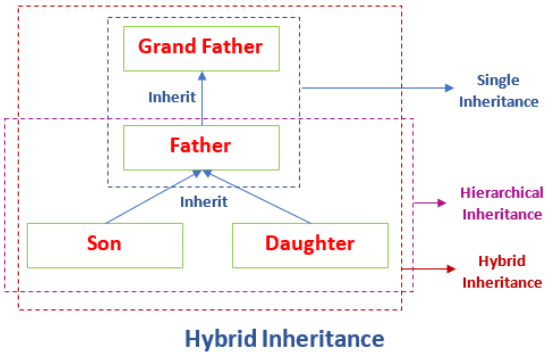
```
    }
    public void use() {
        System.out.println("Warrior uses a sword.");
    }
}
class Mage implements Character, Weapon {
    public void attack() {
        System.out.println("Mage attacks with a wand.");
    }
    public void use() {
        System.out.println("Mage uses a wand.");
    }
}
public class Main {
    public static void main(String[] args) {
        Warrior warrior = new Warrior();
        Mage mage = new Mage();
        warrior.attack(); // Output: Warrior attacks with a sword.
        warrior.use(); // Output: Warrior uses a sword.
        mage.attack(); // Output: Mage attacks with a wand.
        mage.use(); // Output: Mage uses a wand.
    }
}
```

Output:

Warrior attacks with a sword.
Warrior uses a sword.
Mage attacks with a wand.
Mage uses a wand.

Hybrid Inheritance in Java

The hybrid inheritance is the composition of two or more types of inheritance. The main purpose of using hybrid inheritance is to modularize the code into well-defined classes. It also provides the code reusability.



Hybrid Inheritance

The hybrid inheritance can be achieved by using the following combinations:

- *Single and Multiple Inheritance (not supported but can be achieved through interface)*
- *Multilevel and Hierarchical Inheritance*
- *Hierarchical and Single Inheritance*
- *Multiple and Multilevel Inheritance*

Here, we are going to take an example to demonstrate the working of hybrid inheritance in Java.
Example

```
class C
```

```
{
public void disp()
{
System.out.println("C");
}
}
class A extends C
{
public void disp()
{
System.out.println("A");
}
}
class B extends C
{
public void disp()
{
System.out.println("B");
}
}
public class D extends A
{
public void disp()
{
System.out.println("D");
}
public static void main(String args[])
{
D obj = new D();
obj.disp();
}
}
```

Output:

D

1. Why multiple inheritance is not supported in Java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in Java. Suppose there are three classes A, B, and C. The C class inherits A and B classes. If A and B classes have the same method and we call it from child class object, there will be ambiguity to call the method of A or B class. It is called diamond problem.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So, whether you have same method or different, there will be compile time error.

```
class A {
void msg(){System.out.println("Hello");}
}
class B {
void msg(){System.out.println("Welcome");}
}
public class Main extends A,B {
 public static void main(String args[]){
   Main obj = new Main();
```

```
    obj.msg();//Now which msg() method would be invoked?
  }
  }
```

Output:

Compile Time Error

2. How to achieve Multiple Inheritance in Java?

Java supports multiple inheritance through interfaces only, where a class can implement multiple interfaces. Multiple inheritance in Java is not possible by class, but it is possible through interfaces.

Method Overriding in Java

***Method Overriding** in Java is used to achieve runtime polymorphism and dynamic behavior type.*

Rules for Java Method Overriding

The following are the important rules of method overriding in Java:

1. *Same Method Name: .*
2. *Same Parameters: .*
3. *IS-A Relationship (Inheritance): .*
4. *Same Return Type or Covariant Return Type:*
5. *Access Modifier Restrictions:*
6. *No Final Methods.*
7. *No Static Methods:*

Let's understand the problem that we may face in the program if we do not use method overriding.

Example

```
//Java Program to demonstrate why we need method overriding
//Here, we are calling the method of parent class with child class object
//Creating a parent class
class Vehicle{
  void run(){System.out.println("Vehicle is running");}
}
//Creating a child class
class Bike extends Vehicle{}
//Creating a Main class to create object and call methods
public class Main{
  public static void main(String args[]){
  //creating an instance of child class
  Bike obj = new Bike();
  //calling the method with child class instance
  obj.run();
  }
}
```

Output:

Vehicle is running

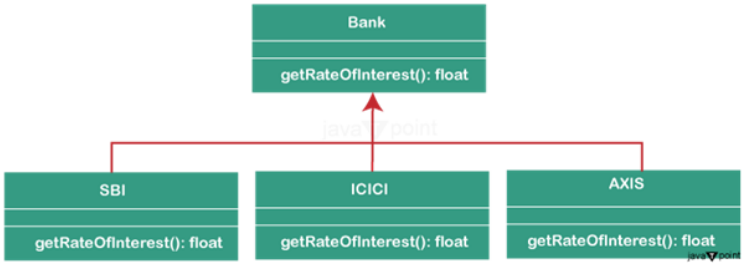
Example

```
//Java Program to illustrate the use of Java Method Overriding
//Creating a parent class.
class Vehicle{
  //defining a method
  void run(){System.out.println("Vehicle is running");}
```

```
}
//Creating a child class
class Bike extends Vehicle{
  //defining the same method as in the parent class
  void run(){System.out.println("Bike is running safely");}
}
//Creating a Main class to create object and call method
public class Main{
  public static void main(String args[]){
  Bike obj = new Bike();//creating object
  obj.run();//calling method
  }
}
```

Output:

Bike is running safely



Example

```
    //Java Program to demonstrate the real scenario of Java Method Overriding
    //where three classes are overriding the method of a parent class.
    //Creating a parent class.
    class Bank{
    int getRateOfInterest(){return 0;}
    }
    //Creating child classes
    class SBI extends Bank{
    int getRateOfInterest(){return 8;}
    }
    class ICICI extends Bank{
    int getRateOfInterest(){return 7;}
    }
    class AXIS extends Bank{
    int getRateOfInterest(){return 9;}
    }
    //Create a Main class to create objects and call the methods
    public class Main{
    public static void main(String args[]){
    SBI s=new SBI();
    ICICI i=new ICICI();
    AXIS a=new AXIS();
    System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
    System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
    System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
    }
    }
```

Output:

SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9

difference between Method Overloading and Method Overriding:

Aspect	***Method Overloading***	***Method Overriding***
Purpose and Intent	*Method overloading is used to increase the readability of the program.*	*Method overriding is used to provide the specific implementation of the method that is already provided by its superclass.*
Relationship between Classes	*Method overloading is performed within class.*	*Method overriding occurs in two classes that have IS-A (inheritance) relationship.*
Parameter Requirements	*In case of method overloading, parameters must be different.*	*In case of method overriding, parameters must be the same.*
Polymorphism Type	*Method overloading is the example of compile-time polymorphism.*	*Method overriding is the example of runtime polymorphism.*
Return Type Constraints	*In Java, method overloading can't be performed by changing the return type of the method only. Return type can be the same or different in method overloading, but you must have to change the parameter.*	*Return type must be the same or covariant in method overriding.*
Access Modifiers	*It can have any access modifier.*	*The overriding method cannot have a more restrictive access modifier than the overridden method.*
Exceptions	*It can throw any exceptions.*	*It throws only checked exceptions declared in the superclass method or its subclasses.*
Invocation	*It resolved at compile-time.*	*It resolved at runtime using dynamic method dispatch.*
Inheritance Required	*Not required.*	*Required (through subclassing or interface implementation).*
Binding	*It uses static binding.*	*It uses dynamic binding.*

Early and Late Binding	*It is also known as early binding.*	*It is also known as late binding.*
Signature Matching	*Method signatures must be different.*	*Method signatures must be identical.*
Statis Method	*Static method can be overloaded.*	*Static method cannot be overridden.*
Final and Public Method	*It is applicable to both private and final methods.*	*It is not applicable to private and final methods.*
Error	*If any error occurs, can be caught at compile-time.*	*If any error occurs, can be caught at run-time.*

Java Access Modifiers with Method Overriding
If you are overriding any method, overridden method (i.e. declared in subclass) must not be more restrictive.
Example
an example to demonstrate the Access Modifiers with Method Overriding in Java.

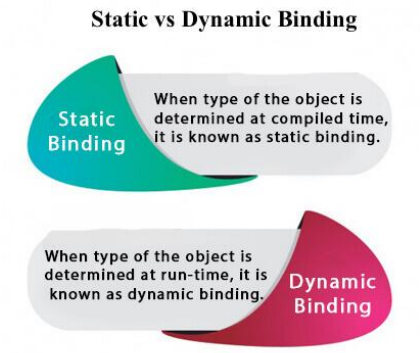
```
class A{
protected void msg(){System.out.println("Hello java");}
}
public class Simple extends A{
void msg(){System.out.println("Hello java");}//C.T.Error
 public static void main(String args[]){
   Simple obj=new Simple();
   obj.msg();
   }
}
```

Output:

```
Simple.java:6: error: msg() in Simple cannot override msg() in A
void msg(){System.out.println("Hello java");}//C.T.Error
     ^
  attempting to assign weaker access privileges; was protected
1 error
```

Static and Dynamic Binding in Java
There are two types of binding in Java:

1. ***Static (Compile-time) Binding:*** *The method to be called is determined at* ***compile time****. It happens with private, final, and static methods and* ***method overloading****.* ***Static binding*** *is also known as* ***Early binding****.*
2. ***Dynamic (Runtime) Binding:*** *The method to be called is determined* ***at runtime*** *based on the* ***actual object****, not the reference type. It happens with* ***method overriding****.* ***Dynamic binding*** *is also known as* ***Late binding****.*



1. variables have a type

Every variable in Java has a type, which can be primitive or non-primitive.

```
int data=30;
```

Here data variable is a type of int.

2. References have a type

A reference variable also has a type, which is the class it can refer to.

```
class Dog{
 public static void main(String args[]){
  Dog d1;//Here d1 is a type of Dog
 }
    }
```

3. Objects have a type

An object is an instance of a particular Java class. However, it is also considered an instance of its superclass.

```
class Animal{}

class Dog extends Animal{
 public static void main(String args[]){
  Dog d1=new Dog();
 }
}
```

Here d1 is an instance of Dog class, but it is also an instance of Animal.

1. Static Binding in Java

When the type of an object is determined at compile time (by the compiler), it is known as static binding. Static binding occurs with private, final, or static methods, as well as with method overloading, because the compiler knows exactly which method to call.

Example of Static Binding

The following example demonstrates the static binding:

```
class Dog{
 private void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Dog d1=new Dog();
  d1.eat();
}
}
```

The following example demonstrates the dynamic binding:

```
class Animal{
 void eat(){System.out.println("animal is eating...");}
}
```

1.

```
class Dog extends Animal{
 void eat(){System.out.println("dog is eating...");}

 public static void main(String args[]){
  Animal a=new Dog();
  a.eat();
 }
2.  }
```

Output:

dog is eating...

Difference Between Static and Dynamic Binding

The following table shows the difference between static and dynamic binding in Java:

Static Binding	***Dynamic Binding***
Static binding occurs at compile time, when the compiler determines which method to call.	*Dynamic binding occurs at run time, when the method to be called is determined based on the actual object.*
It is used with private, final, static methods, and method overloading.	*It is used with overridden methods to achieve runtime polymorphism.*
In static binding, the method call depends on the reference type.	*In dynamic binding, the method call depends on the actual object type.*
Static binding provides compile-time polymorphism.	*Dynamic binding provides runtime polymorphism.*

Polymorphism in Java

Polymorphism in Java is an object-oriented concept that allows the same method name to perform different tasks based on the object or parameters.

The word <**poly** means many, and **morphs** means forms. Therefore, polymorphism means **many forms**.

There are two types of polymorphism in Java:

1. Compile-time Polymorphism
2. Runtime Polymorphism.

We can perform polymorphism in Java by method overloading and method overriding.

1. Java Compile-Time Polymorphism

In Java, **method overloading** is used to achieve **compile-time polymorphism**. Using method overloading, a class can have multiple methods with the same name but different parameter lists. The compiler determines which method to call **at compile time** based on the **number, type, and order of parameters** passed.

In method overloading, methods must have the same name but must differ in the **number or data types of parameters**. When a method is called, the compiler selects the most suitable overloaded method based on the arguments provided. If an exact match is found, that method is invoked; otherwise, the compiler applies **type promotion** to find the closest matching method.

Example of Compile-Time Polymorphism

The following example demonstrates the implementation of compiler-time polymorphism using method overloading:

Example

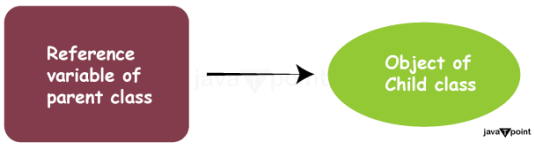
```
class Calculation {
   int add(int a, int b) {
      return a + b;
   }
   double add(double a, double b) {
      return a + b;
   }
}
public class Main {
   public static void main(String[] args) {
      Calculation calc = new Calculation();
      // Compile-
time polymorphism: selecting the appropriate add method based on parameter types
      System.out.println("Sum of integers: " + calc.add(5, 3));
      System.out.println("Sum of doubles: " + calc.add(2.5, 3.7));
   }
}
```

Output:

Sum of integers: 8

Sum of doubles: 6.2

Example:




```
class A{}
class B extends A{}
A a=new B();//upcasting
```

For upcasting, we can use the reference variable of class type or an interface type. For Example:

```
interface I{}
class A{}
class B extends A implements I{}
```

Here, the relationship of B class would be:

B IS-A A

B IS-A I

B IS-A Object

Since Object is the root class of all classes in Java, so we can write B IS-A Object.

Example of Runtime Polymorphism

In this example, we are creating two classes Bike and Splendor. Splendor class extends Bike class and overrides its run() method. We are calling the run method by the reference variable of Parent class. Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.

Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.

Example

```
class Bike{
  void run(){System.out.println("running");}
}
```

```
class Splendor extends Bike{
  void run(){System.out.println("running safely with 60km");}
}
public class Main{
  public static void main(String args[]){
    Bike b = new Splendor();//upcasting
    b.run();
  }
}
```

Output:

running safely with 60km.

Note: This example is also given in method overriding but there was no upcasting.

Source Code

```
class Bank{
   float getRateOfInterest(){return 0;}
}
class SBI extends Bank{
   float getRateOfInterest(){return 8.4f;}
}
class ICICI extends Bank{
   float getRateOfInterest(){return 7.3f;}
}
class AXIS extends Bank{
   float getRateOfInterest(){return 9.7f;}
}
public class Main{
 public static void main(String args[]){
  Bank b;
  b=new SBI();
  System.out.println("SBI Rate of Interest: "+b.getRateOfInterest());
  b=new ICICI();
  System.out.println("ICICI Rate of Interest: "+b.getRateOfInterest());
  b=new AXIS();
  System.out.println("AXIS Rate of Interest: "+b.getRateOfInterest());
 }
     }
```

Output:

SBI Rate of Interest: 8.4

ICICI Rate of Interest: 7.3

AXIS Rate of Interest: 9.7

Example 2: Shape Class

example to demonstrate the shape class using runtime polymorphism.

Source Code

```
class Shape{
   void draw(){System.out.println("drawing...");}
}
class Rectangle extends Shape{
   void draw(){System.out.println("drawing rectangle...");}
}
class Circle extends Shape{
```

```
    void draw(){System.out.println("drawing circle...");}
  }
  class Triangle extends Shape{
    void draw(){System.out.println("drawing triangle...");}
  }
  public class Main{
    public static void main(String args[]){
      Shape s;
      s=new Rectangle();
      s.draw();
      s=new Circle();
      s.draw();
      s=new Triangle();
      s.draw();
    }
  }
```

Output:

drawing rectangle...
drawing circle...
drawing triangle...

```
  class Animal{
    void eat(){System.out.println("eating...");}
  }
  class Dog extends Animal{
    void eat(){System.out.println("eating bread...");}
  }
  class Cat extends Animal{
    void eat(){System.out.println("eating rat...");}
  }
  class Lion extends Animal{
    void eat(){System.out.println("eating meat...");}
  }
  public class Main{
    public static void main(String[] args){
      Animal a;
      a=new Dog();
      a.eat();
      a=new Cat();
      a.eat();
      a=new Lion();
      a.eat();
    }
  }
```

Output:

eating bread...
eating rat...
eating meat...

Example

```
  class Bike{
```

```
 int speedlimit=90;
}
class Honda extends Bike{
 int speedlimit=150;
}
public class Main{
 public static void main(String args[]){
  Bike obj=new Honda();
  System.out.println(obj.speedlimit);//90
 }
}
```

Output:

90

Example

```
class Animal{
   void eat(){System.out.println("eating");}
}
class Dog extends Animal{
   void eat(){System.out.println("eating fruits");}
}
class BabyDog extends Dog{
   void eat(){System.out.println("drinking milk");}
}
public class Main{
   public static void main(String args[]){
      Animal a1,a2,a3;
      a1=new Animal();
      a2=new Dog();
      a3=new BabyDog();
      a1.eat();
      a2.eat();
      a3.eat();
   }
}
```

Output:

eating
eating fruits
drinking Milk

Example

```
class Animal{
   void eat(){System.out.println("animal is eating...");}
}
class Dog extends Animal{
   void eat(){System.out.println("dog is eating...");}
}
class BabyDog extends Dog{}

public class Main{
   public static void main(String args[]){
```

```
        Animal a=new BabyDog();
        a.eat();
    }
}
```

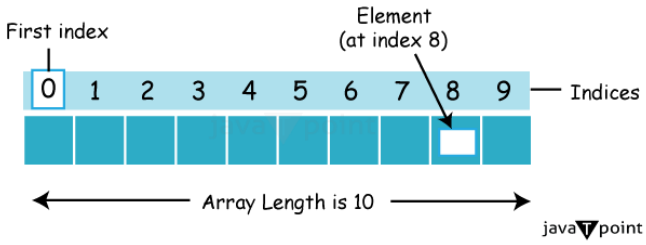
Advantages of Polymorphism
There are several advantages of Polymorphism in Java. Some of them are as follows:
1. Code Reusability
2. Flexibility and Extensibility
3. Dynamic Method Invocation:
4. Interface Implementation:
5. Method Overloading:
6. Reduced Code Complexity:

Array in Java

In Java, an array is an object that stores elements of a similar data type in contiguous memory locations. It is a data structure used to store a fixed number of elements.
Arrays in Java are **index-based**, meaning the first element is at index 0, the second at index 1, and so on.
In Java, an array is considered an object of a dynamically created class. All arrays in Java:

- Implement the Serializable and Cloneable interfaces
- Derive from the Object class

Arrays can store **primitive values** or **objects**, and Java supports both **single-dimensional** and **multi-dimensional** arrays.
Java also supports **anonymous arrays**, which are not available in C/C++. This feature allows us to create and pass arrays without explicitly declaring a variable.
Arrays are of two types, single dimensional and multi-dimensional.



single-Dimensional Array in Java:

A single-dimensional array in Java is a linear collection of elements of the same data type. It can be declared and instantiated using the following syntax.
Syntax of Declaring an Array
Below is the syntax to declare a single dimensional array in Java:

1. dataType[] arr; (or)
2. dataType []arr; (or)
3. dataType arr[];

Syntax of Instantiation of an Array
Below is the syntax to Instantiate a single dimensional array in Java:

1. arrayRefVar=**new** datatype[size];

Example of Single Dimensional Array
Let's see the simple example of java array, where we are going to declare, instantiate, initialize and traverse an array.

Source Code

```
public class Main{
 public static void main(String args[]){
   //declaration and instantiation of an array
   int a[]=new int[5];
   a[0]=10;//initialization
   a[1]=20;
   a[2]=70;
   a[3]=40;
   a[4]=50;
   //traversing array
   for(int i=0;i<a.length;i++){//length is the property of array
      System.out.println(a[i]);
   }
 }
}
```

Output:

10
20
70
40
50

Multidimensional Array (Two-Dimensional Array)

A multidimensional array (two-dimensional array) in Java is an array of arrays, where each element of the main array is itself an array. It is used to store data in a **table-like structure** with rows and columns.

Syntax of Declaring a Two-Dimensional Array

Below is the syntax to declare a two-dimensional array in Java:

1. dataType[][] arrayName;

Syntax of Instantiation of a Two-Dimensional Array

Below is the syntax to instantiate a two-dimensional array in Java:

1. arrayName = **new** dataType[rows][columns];

or you can combine declaration and instantiation:

1. dataType[][] arrayName = **new** dataType[rows][columns];

Example of Two-Dimensional Array

Let's see a simple example of a two-dimensional array in Java, where we declare, instantiate, initialize, and traverse the array:

```
public class Main {
   public static void main(String[] args) {
     // Declare and instantiate a 2D array
     int[][] numbers = new int[2][3];

     // Initialize the array
     numbers[0][0] = 10;
     numbers[0][1] = 20;
     numbers[0][2] = 30;
     numbers[1][0] = 40;
     numbers[1][1] = 50;
     numbers[1][2] = 60;
```

```
        // Traverse and print the array
        for (int i = 0; i < numbers.length; i++) {
            for (int j = 0; j < numbers[i].length; j++) {
                System.out.print(numbers[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

10 20 30
40 50 60

Accessing Array Elements Using For Each Loop

We can access the array elements using **for-each loop**. The Java for-each loop accesses the array elements one by one. It holds an array element in a variable, then executes the body of the loop.

Syntax

The syntax of the for-each loop is given below:

```
1. for(data_type variable:array){
2. //body of the loop
3. }
```

Let's see the example of printing the elements of the Java array using the for-each loop.

Example

```
//Java Program to print the array elements using for-each loop
public class Main{
 public static void main(String args[]){
   //declaration and initialization of an array
   int arr[]={33,3,4,5};
   //traversal of an array using for-each loop
   for(int i:arr)
      System.out.println(i);
 }
}
```

Output:

33
3
4
5

Passing Array to a Method

We can pass the Java array to the method so that we can reuse the same logic on any array. When we pass an array to a method in Java, we are essentially passing a reference to the array. It means that the method will have access to the same array data as the calling code, and any modifications made to the array within the method will affect the original array.

Let's see the simple example to get the minimum number of an array using a method.

Example

```
//Java Program to demonstrate the way of passing an array to method.
public class Main{
//creating a method which receives an array as a parameter
   static void min(int arr[]){
      int min=arr[0];
      for(int i=1;i<arr.length;i++)
         if(min>arr[i])
```

```
            min=arr[i];

        System.out.println(min);
      }

     public static void main(String args[]){
       int a[]={33,3,4,5};//declaring and initializing an array
       min(a);//passing array to method
     }
    }
```

Output:

```
    3
    //Java Program to demonstrate the way of passing an anonymous array to method
    public class Main{
      //creating a method which receives an array as a parameter
      static void printArray(int arr[]){
        for(int i=0;i<arr.length;i++)
          System.out.println(arr[i]);
      }
     public static void main(String args[]){
      printArray(new int[]{10,22,44,66});//passing anonymous array to method
     }
    }
```

Output:

```
    10
    22
    44
    66
```

Example

```
    //Java Program to return an array from the method
    public class Main{
      //creating method which returns an array
      static int[] get(){
        return new int[]{10,30,50,90,60};
      }
     public static void main(String args[]){
      //calling method which returns an array
      int arr[]=get();
      //printing the values of an array
      for(int i=0;i<arr.length;i++)
        System.out.println(arr[i]);
     }
    }
```

Output:

```
    10
    30
    50
    90
    60
```

The following example demonstrates ArrayIndexOutOfBoundsException in Java array:

```
//Java Program to demonstrate the case of
//ArrayIndexOutOfBoundsException in a Java Array.
public class TestArrayException{
public static void main(String args[]){
int arr[]={50,60,70,80};
for(int i=0;i<=arr.length;i++){
System.out.println(arr[i]);
}
}}
```

Output:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 4
at TestArrayException.main(TestArrayException.java:5)
50
60
70
80
```

Jagged Arrays

In Java, a jagged array is an array of arrays where each row of the array can have a different number of columns. This contrasts with a regular two-dimensional array, where each row has the same number of columns.

Declaring and Initializing a Jagged Array

To declare a jagged array in Java, you first declare the array of arrays, and then you initialize each row separately with its own array of columns.

```
1. int arr[][] = new int[3][];
2. arr[0] = new int[3];
3. arr[1] = new int[4];
4. arr[2] = new int[2];
```

Example

The following example demonstrates the use of jagged array in Java:

Example

```
//Java Program to illustrate the jagged array
public class Main{
   public static void main(String[] args){
      //declaring a 2D array with odd columns
      int arr[][] = new int[3][];
      arr[0] = new int[3];
      arr[1] = new int[4];
      arr[2] = new int[2];
      //initializing a jagged array
      int count = 0;
      for (int i=0; i<arr.length; i++)
         for(int j=0; j<arr[i].length; j++)
            arr[i][j] = count++;

      //printing the data of a jagged array
      for (int i=0; i<arr.length; i++){
         for (int j=0; j<arr[i].length; j++){
            System.out.print(arr[i][j]+" ");
         }
         System.out.println();//new line
      }
```

```
    }
  }
```
Output:

0 1 2
3 4 5 6
7 8

```
//Java Program to get the class name of array in Java
  public class Main{
   public static void main(String args[]){
     //declaration and initialization of array
     int arr[]={4,4,5};
     //getting the class name of Java array
     Class c=arr.getClass();
     String name=c.getName();
     //printing the class name of Java array
     System.out.println(name);
   }
  }
```
Output:

[I

example demonstrates how to copy elements from one array to another in Java using the System.arraycopy() method.

Example

```
  //Java Program to copy a source array into a destination array in Java
  public class Main {
     public static void main(String[] args) {
        //declaring a source array
        char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
              'i', 'n', 'a', 't', 'e', 'd' };
        //declaring a destination array
        char[] copyTo = new char[7];
        //copying array using System.arraycopy() method
        System.arraycopy(copyFrom, 2, copyTo, 0, 7);
        //printing the destination array
        System.out.println(String.valueOf(copyTo));
     }
  }
```
Output:

caffein

```
  //Java Program to clone the array
  class TestarrayClone{
  public static void main(String args[]){
  int arr[]={33,3,4,5};
  System.out.println("Printing original array:");
  for(int i:arr)
  System.out.println(i);
  System.out.println("Printing clone of the array:");
  int carr[]=arr.clone();
  for(int i:carr)
```

```
System.out.println(i);
System.out.println("Are both equal?");
System.out.println(arr==carr);
}
}
```

Output:

```
Printing original array:
33
3
4
5
Printing clone of the array:
33
3
4
5
Are both equal?
False
```

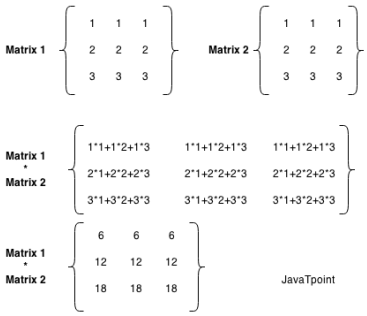
Example

Let's see a simple example that adds two matrices.

```
//Java Program to demonstrate the addition of two matrices in Java
class ArrayAddition{
public static void main(String args[]){
//creating two matrices
int a[][]={{1,3,4},{3,4,5}};
int b[][]={{1,3,4},{3,4,5}};

//creating another matrix to store the sum of two matrices
int c[][]=new int[2][3];

//adding and printing addition of 2 matrices
for(int i=0;i<2;i++){
for(int j=0;j<3;j++){
c[i][j]=a[i][j]+b[i][j];
System.out.print(c[i][j]+" ");
}
System.out.println();//new line
}

}}
```

Output:

```
2 6 8
6 8 10
```



Example

The following example demonstrates multiplication of two matrices:

```
//Java Program to multiply two matrices
public class MatrixMultiplicationExample{
public static void main(String args[]){
//creating two matrices
int a[][]={{1,1,1},{2,2,2},{3,3,3}};
int b[][]={{1,1,1},{2,2,2},{3,3,3}};

//creating another matrix to store the multiplication of two matrices
int c[][]=new int[3][3];  //3 rows and 3 columns

//multiplying and printing multiplication of 2 matrices
for(int i=0;i<3;i++){
for(int j=0;j<3;j++){
c[i][j]=0;
for(int k=0;k<3;k++)
{
c[i][j]+=a[i][k]*b[k][j];
}//end of k loop
System.out.print(c[i][j]+" ");  //printing matrix element
}//end of j loop
System.out.println();//new line
}
}}
```

Output:

6 6 6
12 12 12
18 18 18

Abstract class in Java:

Abstract class are used to define structure and behaviour of classes with in an inheritance hierarchy. They act as a blue print of other classes where some methods may be declared without implementation.

Abstraction is a process of hiding implementation details and showing only the required functionality. To the user.

There are two ways :

Using abstraction class (can provide 0 to 100% abstraction)

Using interface(provides 100% abstraction).

Points to remember:

- An abstract class must be declared in abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods wh9ich will force the subclass not to change the body of the method.

Syntax:

```
public abstract class Shape {
   public abstract double area();
   public void display() {
      System.out.println("This is a shape.");
   }
}
```

Abstract Method:
A method that is declared using the **abstract** keyword and does not have an implementation (method body) is known as an **abstract method**.

Example:
abstract void printStatus(); *// no method body*

An abstract method only declares the method signature. Its implementation must be provided by the subclass that extends the abstract class.

Abstract Class having Constructor, Data Members, and Methods

An abstract class in Java can contain **data members, abstract methods, concrete methods, constructors**, and even the main() method.

Example:

- A **constructor** that runs when an object of the subclass is created.
- An **abstract method** run() that must be implemented by subclasses.
- A **concrete method** changeGear() that can be inherited as-is.

```
//Example of an abstract class that has abstract and non-abstract methods
 abstract class Bike{
 Bike(){
System.out.println("bike is created");}
 abstract void run();
 void changeGear(){System.out.println("gear changed");}
}

//Creating a Child class which inherits Abstract class
class Honda extends Bike{
 void run(){System.out.println("running safely..");}
}
```

```
//Creating a Main class which calls abstract and non-abstract methods
public class Main{
 public static void main(String args[]){
  Bike obj = new Honda();
  obj.run();
  obj.changeGear();
 }
}
```

Abstract Class	Interface
Abstract classes can have abstract and non-abstract methods.	An interface can have only abstract methods. Since Java 8, it can have default and static methods also.
Abstract class does not support multiple inheritance.	Interface supports multiple inheritance.
Abstract class can have final, non-final, static and non-static variables.	An interface has only static and final variables.
Abstract class can provide the implementation of the interface.	An interface cannot provide the implementation of an abstract class.
The abstract keyword is used to declare an abstract class.	The interface keyword is used to declare an interface.
An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
An abstract class can be extended using the keyword "extends".	An interface can be implemented using the keyword "implements".
A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.

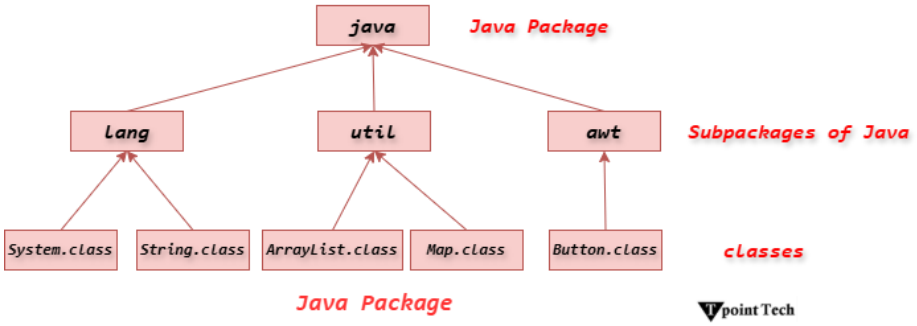
Example:
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();

Packages in Java

A **Java package** is a group of similar types of classes, interfaces and sub-packages. This chapter explains how Java packages work and how they help in organizing, managing, and using classes in Java programs.

There are two types of Java Packages:

- Built-in Packages
- User-defined Packages



Access Modifiers in Java

Access modifiers in Java are keywords that define where a class, method, or variable can be accessed. These modifiers help to protect data and control access to code.

There are four types of access modifiers in Java

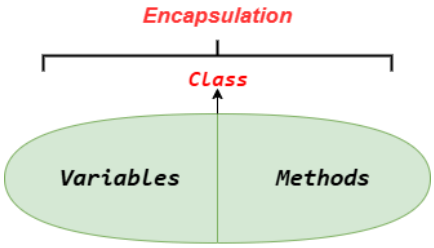
Public
Private
protected
default
Accessibility or Scope of Java Access Modifiers

The following table shows the accessibility or scope of Java access modifiers:

Access Modifier	Within Class	Within Package	Outside Package by Subclass Only	Outside Package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Encapsulation in Java

Encapsulation in Java is an important concept in object-oriented programming that helps protect data and restrict direct access to class members.



Encapsulation in Java

The following are the important reasons to use the encapsulation:

- Encapsulation helps to protect data and control access to it.
- It protects sensitive data from being access directly.
- It hides unnecessary data from the user of a class, and only shows the functionality of a class.
- Changes can be made internally without affecting the external interface.
- It is easier to scale applications because it provides flexibility to add or modify features without impacting the entire codebase.

Encapsulated classes can be reused across projects.

```
//A Java class which is a fully encapsulated class.
//It has a private data member and getter and setter methods.
package com.tpointtech;
public class Student{
//private data member
private String name;
//getter method for name
public String getName(){
return name;
}
//setter method for name
public void setName(String name){
this.name=name
}
}
```

Exception Handling in Java

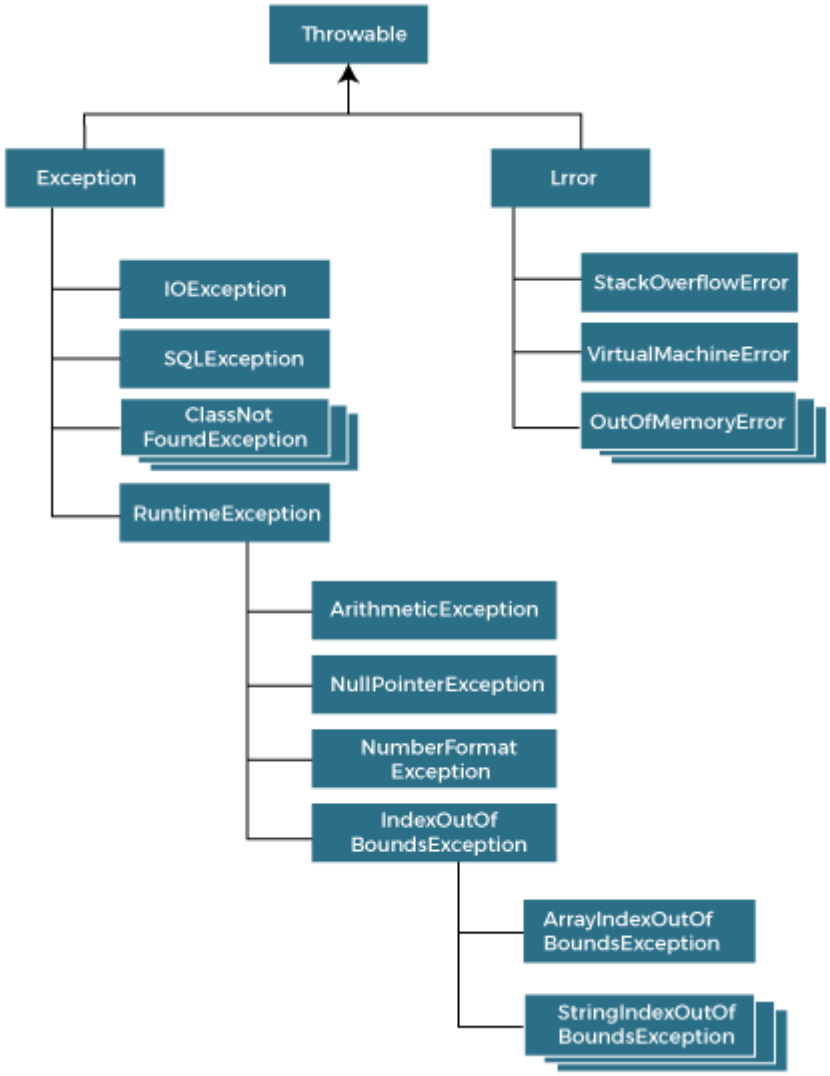
Exception Handling in Java is one of the most powerful mechanisms to handle runtime errors so that the normal flow of the application can be maintained.

Hierarchy of Java Exception classes

Exceptions are represented by classes, and all exception classes are part of the **java.lang package**. They are organized in a hierarchical structure with **Throwable** at the top. The main hierarchy is as follows:

- **Throwable**: The superclass of all errors and exceptions in Java.
    - **Error**: Represents serious system errors that applications usually cannot handle (e.g., OutOfMemoryError).
    - **Exception**: Represents conditions that a program might want to catch.
        - **Checked Exceptions**: Must be either **handled using try-catch** or **declared using throws** (e.g., IOException, SQLException).
        - **Unchecked Exceptions (RuntimeException)**: Do **not need explicit handling**, typically caused by programming errors (e.g., NullPointerException, ArithmeticException).

The following diagram demonstrates the same:

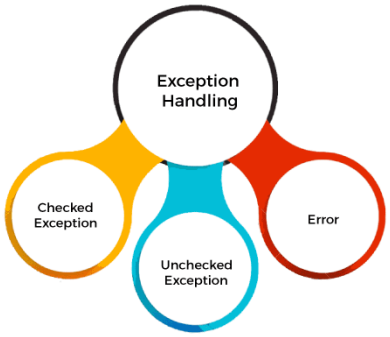


Types of Java Exceptions

The exceptions are categorized into two main types: checked exceptions and unchecked exceptions. Additionally, there is a third category known as errors. Let's delve into each of these types:

- **Checked Exception:** Exceptions that must be handled or declared in the code.

- **Unchecked Exception:** Exceptions that do not require explicit handling and usually occur due to programming errors.
- **Error:** Serious problems that are not meant to be caught by applications, usually system-related.



Java Checked Exceptions

Checked exceptions are the exceptions that are checked at compile-time. This means that the compiler verifies that the code handles these exceptions either by catching them or declaring them in the method signature using the throws keyword. Examples of checked exceptions include:

- **IOException:**An exception is thrown when an input/output operation fails, such as when reading from or writing to a file.
- **SQLException:**It is thrown when an error occurs while accessing a database.
- **ParseException:**Indicates a problem while parsing a string into another data type, such as parsing a date.
- **ClassNotFoundException:**It is thrown when an application tries to load a class through its string name using methods like Class.forName(), but the class with the specified name cannot be found in the classpath.

Example

The following example demonstrates a checked exception using IOException:

```
import java.io.File;

import java.io.FileReader;

import java.io.IOException;

public class CheckedExceptionExample {
   public static void main(String[] args) {
      try {
         File file = new File("example.txt");
         FileReader fr = new FileReader(file); // May throw IOException
         System.out.println("File opened successfully.");
         fr.close();
      } catch (IOException e) {
         System.out.println("An IOException occurred: " + e.getMessage());
      }
```

```
    }
}
```

Output:

An IOException occurred: example.txt (No such file or directory)

Explanation:

- FileReader may throw an **IOException**, which is a **checked exception**.
- The exception is handled using a try-catch block.

Java Unchecked Exceptions (Runtime Exceptions)

Unchecked exceptions, also known as runtime exceptions, are not checked at compile-time. These exceptions usually occur due to programming errors, such as logic errors or incorrect assumptions in the code. They do not need to be declared in the method signature using the throws keyword, making it optional to handle them. Examples of unchecked exceptions include:

- **NullPointerException:**It is thrown when trying to access or call a method on an object reference that is null.
- **ArrayIndexOutOfBoundsException:**It occurs when we try to access an array element with an invalid index.
- **ArithmeticException:**It is thrown when an arithmetic operation fails, such as division by zero.
- **IllegalArgumentException:**It indicates that a method has been passed an illegal or inappropriate argument.

Example

The following example demonstrates unchecked exceptions:

```
public class UncheckedExceptionExample {
    public static void main(String[] args) {
        // Example 1: NullPointerException
        String str = null;
        try {
            System.out.println(str.length()); // May throw NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        }

        // Example 2: ArrayIndexOutOfBoundsException
        int[] arr = {1, 2, 3};
        try {
            System.out.println(arr[5]); // Invalid index
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: " + e.getMessage());
        }


        // Example 3: ArithmeticException
        try {
            int result = 10 / 0; // Division by zero
```

```
    } catch (ArithmeticException e) {
      System.out.println("Caught ArithmeticException: " + e.getMessage());
    }
  }
}
```

Output:

Caught NullPointerException: null

Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3

Caught ArithmeticException: / by zero

Java Errors

Errors represent exceptional conditions that are not expected to be caught under normal circumstances. They are typically caused by issues outside the control of the application, such as system failures or resource exhaustion. Errors are not meant to be caught or handled by application code. Examples of errors include:

- **OutOfMemoryError:**It occurs when the Java Virtual Machine (JVM) cannot allocate enough memory for the application.
- **StackOverflowError:**It is thrown when the stack memory is exhausted due to excessive recursion.
- **NoClassDefFoundError:**It indicates that the JVM cannot find the definition of a class that was available at compile-time.

Example

The following example demonstrates errors in Java:

```
public class ErrorExample {
  // Example 1: StackOverflowError
  public static void recursiveCall() {
    recursiveCall(); // Infinite recursion
  }

  public static void main(String[] args) {
    try {
      recursiveCall();
    } catch (StackOverflowError e) {
      System.out.println("Caught StackOverflowError: " + e.getMessage());
    }

    // Example 2: OutOfMemoryError
    try {
      int[] largeArray = new int[Integer.MAX_VALUE]; // Request too much memory
    } catch (OutOfMemoryError e) {
      System.out.println("Caught OutOfMemoryError: " + e.getMessage());
    }
  }
}
```

Output:

Caught StackOverflowError: null

Caught OutOfMemoryError: Java heap space

Explanation:

- Errors indicate serious problems that applications generally cannot recover from.
- Unlike checked and unchecked exceptions, handling errors is optional, and they are usually left to the JVM.
- Examples show **StackOverflowError** caused by recursion and **OutOfMemoryError** caused by excessive memory allocation.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

The try-catch Block

The try-catch block is the primary mechanism for handling exceptions. The **try block** contains code that may throw an exception, and the **catch block** handles that exception.

Syntax

The syntax to handle an expectation using try-catch block is:

```
try {
   // Code that may throw an exception
} catch (ExceptionType e) {
```

```
    // Code to handle the exception
}
```
Example

Consider the below example, demonstrating handling an exception using try-catch block:

```
// Java Program to understand the
// use of exception handling
public class Main {
    public static void main(String args[]) {
        try {
            int data = 100 / 0; // May raise ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println(e); // Handling the exception
        }
        System.out.println("rest of the code...");
    }
}
```
Output:

java.lang.ArithmeticException: / by zero

rest of the code...

The finally Block

The finally block is executed regardless of whether an exception occurs or not. It is commonly used to release resources like files or database connections.

Syntax

Here is the syntax to use finally block along with the try-catch block:

```
try {
    // Code that may throw an exception
} catch (Exception e) {
    // Exception handling code
} finally {
    // Cleanup code
}
```
Example

The following example demonstrates the use of the finally block:

```
//Java Program to illustrate the use of finally block
public class Main {
    public static void main(String[] args) {
        try {
            int data = 25 / 0; // May throw ArithmeticException
            System.out.println("Result: " + data);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        } finally {
            System.out.println("This block always executes.");
    1.      }
```

```
        System.out.println("Program continues after finally block...");
    }
}
```

Output:

Error: Division by zero is not allowed.

This block always executes.

Program continues after finally block...

Multiple Catch Block in Java

In Java, multiple catch blocks allow us to handle different types of exceptions separately. This is useful when a single try block contains code that may throw different types of exceptions.

*A **try** block can be followed by one or more catch blocks, and each catch block must handle a different type of exception. This allows you to perform specific actions depending on the type of exception that occurs.*

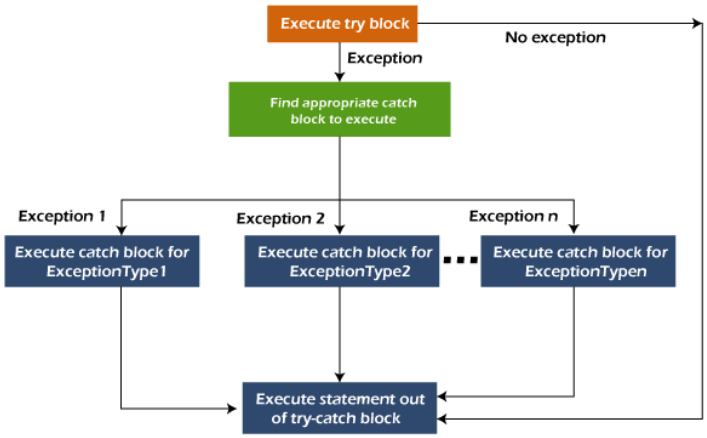
Alternatively, starting from Java 7, we can combine multiple exceptions in a single catch block using the | (pipe) symbol.

Using Multiple Catch Blocks

A multiple catch block is used when a try block can throw different types of exceptions. Each catch handles a specific exception, and more specific exceptions should come before general ones. From Java 7 onward, multiple exceptions can be handled in a single catch block using the | (pipe) symbol.

Flowchart

The following image shows the flowchart of the working of multiple catch blocks:



Syntax

The syntax for using multiple catch blocks:

```
try {
   // Code that may throw multiple exceptions
        } catch (FirstExceptionType e) {
```

```
    // Handle first type of exception
} catch (SecondExceptionType e) {
    // Handle second type of exception
} catch (Exception e) {
    // Handle any other exceptions
}
```

Nested Try Block in Java

In Java, using a try block inside another try block is permitted. It is called as nested try block. Every statement that we enter a statement in try block, context of that exception is pushed onto the stack. For example, the inner try block can be used to handle ArrayIndexOutOfBoundsException, while the outer try block can handle the ArithmeticException (division by zero).

Syntax of Nested Try-Catch

A **nested try block** is a try block placed **inside another try block**. The outer try block can handle exceptions not caught by the inner try block(s).

Here is the syntax to use nested try blocks:

```
try {
    // Outer try block
    // Code that may throw an exception

    try {
        // Inner try block
        // Code that may throw a different exception
    } catch (ExceptionType1 e1) {
        // Inner catch block: handles ExceptionType1
    }

    // Optional: more inner try-catch blocks can be added
} catch (ExceptionType2 e2) {
    // Outer catch block: handles exceptions not caught by inner try
}
```

Difference Between throw and throws Keywords

The following table shows the key differences between difference between throw and throws keywords:

Characteristics	throw	throws
Definition	Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
Declaration	It is used inside a method or block.	It is used with method signature.
Usage	Type of exception using throw keyword, we can only propagate unchecked exception i.e., the	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used

	checked exception cannot be propagated using throw only.	to propagate checked exceptions only.
Purpose	It triggers the creation and throwing of an exception.	It informs the caller of the method about possible exceptions may occur.
Syntax	throw new ExceptionType("message");	public void myMethod() throws IOException
Number of Exceptions	It can throw only one exception at a time.	It can declare multiple exceptions separated by commas.
Followed By	An instance of Throwable (or subclass).	One or more exception classes (checked exception only).
Internal Implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

final Keyword

A final is a keyword used to restrict changes. A final variable cannot be reassigned, a final method cannot be overridden, and a final class can't be subclassed. It ensures immutability and is often used for constants.
Characteristics of final Keyword
There are several characteristics of the final keyword.

- **Used to declare constants:** A final variable's value cannot be changed once assigned.
- **Prevents method overriding:** A final method cannot be overridden in a subclass.
- **Always executes**: It runs regardless of whether an exception is thrown or caught.

finally Block

A finally is a block used with try-catch statements to execute important code like closing resources. It always runs whether an exception occurs or not, ensuring cleanup actions are performed.
Characteristics of finally
There are several characteristics of the finally method in Java.

- ***Part of exception handling**: It is used with try and catch blocks.*
- ***Always executes**: It runs regardless of whether an exception is thrown or caught.*
- ***Used for resource cleanup**: Like closing files, database connections, etc.*

finalize() Method

A finalize() is a method called by the garbage collector before an object is destroyed. It's used to perform cleanup operations like releasing resources, but it is deprecated and not recommended in modern Java.

Characteristics of finalize() Method

There are several characteristics of the finalize() method.

- **Defined in java.lang.Object class:** Can be overridden by any class.
- **Called by the garbage collector:** Just before destroying an object.
- **Used for cleanup:** Freeing resources like file handles or network sockets.
- **Not reliable:** No guarantee it will be executed timely or at all.

A list of differences between final, finally and finalize is given below:

Aspect	final	finally	finalize()
Definition	final is the keyword and access modifier that is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java that is used to perform cleanup processing just before an object is garbage collected.
Applicable to	The final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	The finalize() method is used with the objects.
Functionality	(1) Once declared, a final variable becomes a constant and cannot be modified. (2) The final method cannot be overridden by sub class. (3) The final class cannot be inherited.	(1) finally block runs the important code even if an exception occurs or not. (2) The finally block cleans up all the resources used in a try block	The finalize() method performs the cleaning activities with respect to the object before its destruction.
Execution	The final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. Its execution is not dependant on the exception.	The finalize() method is executed just before the object is destroyed.
Inheritance/Overriding	Prevents method overriding and class inheritance.	Not related to inheritance or overriding.	Can be overridden from the Object class.
Use Case	To create constants, prevent subclassing or method overriding.	To ensure resource cleanup, like closing files, streams, etc., in exception handling.	To perform cleanup activities like memory/resource release before the object is collected.

Control	Gives compile-time control to avoid unintended changes.	Provides runtime control to handle cleanup post-exception or normal flow.	Invoked by JVM (Java Virtual Machine); the programmer cannot call it directly for cleanup.
Example	final int x=10;	Finally{closeResource	protected void finalize

---

Custom (User-Defined) Exception in Java

A custom (user-defined) exception is an exception created by the programmer to handle application-specific error conditions. These exceptions are implemented by creating a class that extends either the **Exception** class (for checked exceptions) or the **RuntimeException** class (for unchecked exceptions).
Custom exceptions allow developers to define meaningful error messages and handle errors in a way that better matches the program's requirements.
Syntax
Here is the syntax to define a custom exception:

```
class CustomException extends Exception {
   CustomException(String message) {
      super(message);
   }
}
```

The following are a few of the reasons to use custom exceptions:

- It represents application-specific errors.
- To catch and provide specific treatment to a subset of existing Java exceptions.
- **Business Logic Exceptions:** These are the exceptions related to business logic and workflow. It is useful for users and developers to understand the exact problem in a meaningful way.
- It provides clear, descriptive error messages for better debugging.

Creating a Custom Exception
To create a custom exception:

1. You need to extend the **Exception class** (for checked exceptions) or **RuntimeException** class (for unchecked exceptions).
2. After that, you need to initialize the exception with custom messages using the constructor.
3. Optionally, you can also add methods to provide additional details about the exception.

Example 1: Custom Exception with Message
In this example, we create a custom exception InvalidAgeException that is thrown when the age is less than 18. The exception message is passed to the parent Exception class using super().

```
// Custom exception class
class InvalidAgeException extends Exception {
   public InvalidAgeException(String message) {
      super(message); // Pass message to parent Exception class
   }
}

// Class that uses the custom exception
public class Main {
   // Method to validate age
   static void validate(int age) throws InvalidAgeException {
      if (age < 18) {
```

```
            throw new InvalidAgeException("Age is not valid to vote");
        } else {
            System.out.println("Welcome to vote");
        }
    }

    public static void main(String[] args) {
        try {
            validate(13); // Invalid age
        } catch (InvalidAgeException ex) {
            System.out.println("Caught the exception");
            System.out.println("An exception occurred: " + ex);
        }
        System.out.println("Rest of the code...");
    }
}
```

Output:

Caught the exception
An exception occurred: InvalidAgeException: Age is not valid to vote
Rest of the code...

Inner Classes (Nested Classes) in Java

Java inner classes are classes defined within another class that are used to improve encapsulation and code organization.

Creating Inner Class

An inner class is created by defining a class inside another class. Here are the steps to create an inner class:

Define the outer class as usual.
Inside the outer class, define the inner class.
Access the inner class using an object of the outer class.

Syntax

Here is the syntax to create an inner class:

```
class OuterClass {
    // Outer class members

    class InnerClass {
        // Inner class members
    }
}
```

Example

In this example, we have an outer class **Student** with an inner class **Address**. The inner class stores the student's address and displays it along with the student's name.

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    // Inner class
    class Address {
```

```
        String city, state;

        Address(String city, String state) {
            this.city = city;
            this.state = state;
        }

        void display() {
            System.out.println("Student Name: " + name);
            System.out.println("City: " + city);
            System.out.println("State: " + state);
        }
    }
}

public class TestStudent {
    public static void main(String[] args) {
        // Create outer class object
        Student student = new Student("Rahul");

        // Create inner class object
        Student.Address address = student.new Address("Delhi", "Delhi");
        address.display();
    }
}
```

Output:

Student Name: Rahul
City: Delhi
State: Delhi

Advantages of Using Inner Classes

Advantages of Using Inner Classes

The following are the main two advantages of using inner classes:

- Inner classes can access all members (fields and methods) of the outer class, including private members that allows tight integration between the inner and outer class.
- Inner classes help logically group related classes in one place that makes the code easier to read and maintain.

Types of Nested Classes

Nested classes are classified into two main types: **non-static nested classes** and **static nested classes**. The **non-static nested classes** are commonly referred to as **inner classes**.

1. Non-Static Nested Class (Inner Class)

Inner classes are non-static classes defined within another class. They are further categorized into:

1. Member Inner Class: A class defined **inside a class but outside any method**.
2. Anonymous Inner Class: A class without a name, typically used **to implement an interface or extend a class**. The compiler automatically assigns a name.
3. Local Inner Class: A class **defined inside a method**, available only within that method.

2. Static Nested Class

A static nested class is a static class defined within another class. It can access only static members of the outer class directly.

Nested Interface

An interface defined inside a class or another interface. Nested interfaces are often used to logically group related interfaces.

Java Anonymous inner class

Java anonymous inner class is an inner class without a name and for which only a single object is created. An anonymous inner class can be useful when making an instance of an object with certain "extras" such as overloading methods of a class or interface, without having to actually subclass a class.
In simple words, a class that has no name is known as an anonymous inner class. It should be used if you have to override a method of class or interface.
Different Ways to Create Anonymous Inner Class
An anonymous inner class is a class without a name. It can be created in two ways:

1. **Using a class (abstract or concrete):**
   The anonymous class **extends an existing class** and provides an implementation for its methods without creating a separate named subclass. It is often used for **short-term method overrides**.
2. **Using an interface:**
   The anonymous class **implements an interface** and provides concrete implementations for its methods. This is commonly used for **callbacks, event handling, or single-use interface implementations.**

Creating Anonymous Inner Class Using a Class
An anonymous inner class can be created by extending a class (abstract or concrete) without giving it a name. It allows you to override methods of the class in a concise way without creating a separate subclass.
Example
The following example demonstrates creating anonymous inner class using a class:

```
abstract class Person{
  abstract void eat();
}
class TestAnonymousInner{
 public static void main(String args[]){
  Person p=new Person(){
  void eat(){System.out.println("nice fruits");}
  };
  p.eat();
 }
}
```

Output:
nice fruits
Creating Anonymous Inner Class Using an Interface
An anonymous inner class can also be created by implementing an interface. It provides the implementation of the interface's methods without creating a separate class.
Example
The following example demonstrates creating anonymous inner class using an interface:

```
interface Eatable{
 void eat();
}
class TestAnnonymousInner1{
 public static void main(String args[]){
 Eatable e=new Eatable(){
```

```
    public void eat(){System.out.println("nice fruits");}
     };
     e.eat();
     }
    }
```

Output:

nice fruits

Multithreading in Java

Multithreading in Java allows multiple threads to run concurrently within a single program. In this chapter, we will learn the concepts, benefits, and implementation of multithreading.

What is Multithreading in Java?

***Multithreading** is a process of executing multiple threads simultaneously within a single program. A **thread** is the smallest unit of execution in a program, and **multithreading** allows multiple threads to share the same memory and resources while running concurrently. This improves the performance of programs by making better use of CPU resources and enables tasks such as background processing, parallel computation, and responsive user interfaces.*

In Java, multithreading can be implemented by:

1. *Extending the Thread class*
2. *Implementing the Runnable interface*

Multithreading Using Extending the Thread Class

*You can create a thread by extending the **Thread** class and overriding its **run()** method. The **run()** method contains the code that defines the thread's task. After creating an object of the subclass, you start the thread using the **start()** method.*

Syntax

The syntax to implement multithreading using extending the Thread class is as follows:

```
class MyThread extends Thread {
   public void run() {
      // Code to be executed in the thread
   }
}

public class TestThread {
   public static void main(String[] args) {
      MyThread t1 = new MyThread();
      t1.start(); // Starts the thread
           }

        }
```

Multithreading by Implementing the Runnable Interface

*Another way to create a thread is by implementing the Runnable interface and providing an implementation for the **run()** method. You then pass the Runnable object to a Thread object and call **start()**. This approach can be used when your class is already extending another class.*

Syntax

The syntax to implement multithreading by implementing the Runnable interface is as follows:

```
class MyRunnable implements Runnable {
   public void run() {
      // Code to be executed in the thread
   }
}

public class TestRunnable {
   public static void main(String[] args) {
      MyRunnable r = new MyRunnable();
      Thread t1 = new Thread(r);
      t1.start(); // Starts the thread
   }
}
```

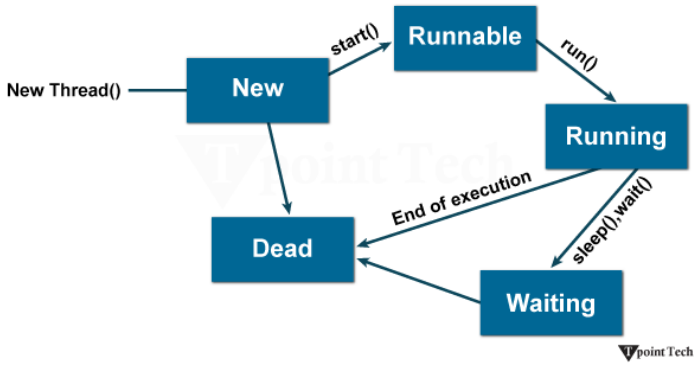
Example

The following example demonstrates creating a thread by implementing Runnable:

```
class MyRunnable implements Runnable {
   public void run() {
      for(int i = 1; i <= 5; i++) {
         System.out.println("Runnable Thread: " + i);
      }
   }
}

public class RunnableExample {
   public static void main(String[] args) {
      MyRunnable r = new MyRunnable();
      Thread t1 = new Thread(r);
      t1.start();
   }
}
```

Output:

```
Thread running: 1
Thread running: 2
Thread running: 3
Thread running: 4
Thread running: 5
```



Thread Methods

The following are the Thread class methods used for multithreading:

Modifier and Type	***Method***	***Description***
void	*start()*	*It is used to start the execution of the thread.*
void	*run()*	*It is used to do an action for a thread.*
static void	*sleep()*	*It sleeps a thread for the specified amount of time.*
static Thread	*currentThread()*	*It returns a reference to the currently executing thread object.*
void	*join()*	*It waits for a thread to die.*

int	*getPriority()*	*It returns the priority of the thread.*
void	*setPriority()*	*It changes the priority of the thread.*
String	*getName()*	*It returns the name of the thread.*
void	*setName()*	*It changes the name of the thread.*
long	*getId()*	*It returns the ID of the thread.*
boolean	*isAlive()*	*It tests if the thread is alive.*
static void	*yield()*	*It causes the currently executing thread object to pause and allow other threads to execute temporarily.*
void	*suspend()*	*It is used to suspend the thread.*
void	*resume()*	*It is used to resume the suspended thread.*

void	*stop()*	*It is used to stop the thread.*
void	*destroy()*	*It is used to destroy the thread group and all of its subgroups.*
boolean	*isDaemon()*	*It tests if the thread is a daemon thread.*
void	*setDaemon()*	*It marks the thread as a daemon or a user thread.*
void	*interrupt()*	*It interrupts the thread.*
boolean	*isinterrupted()*	*It tests whether the thread has been interrupted.*
static boolean	*interrupted()*	*It tests whether the current thread has been interrupted.*
static int	*activeCount()*	*It returns the number of active threads in the current thread's thread group.*

void	*checkAccess()*	*It determines if the currently running thread has permission to modify the thread.*
static boolean	*holdLock()*	*It returns true if and only if the current thread holds the monitor lock on the specified object.*
static void	*dumpStack()*	*It is used to print a stack trace of the current thread to the standard error stream.*
StackTraceElement[]	*getStackTrace()*	*It returns an array of stack trace elements representing the stack dump of the thread.*
static int	*enumerate()*	*It is used to copy every active thread's thread group and its subgroup into the specified array.*

Thread.State	*getState()*	*It is used to return the state of the thread.*
ThreadGroup	*getThreadGroup()*	*It is used to return the thread group to which this thread belongs*
String	*toString()*	*It is used to return a string representation of this thread, including the thread's name, priority, and thread group.*
void	*notify()*	*It is used to give a notification for only one thread that is waiting for a particular object.*
void	*notifyAll()*	*It is used to give a notification to all waiting threads of a particular object.*
void	*setContextClassLoader()*	*It sets the context ClassLoader*

		for the Thread.
ClassLoader	*getContextClassLoader()*	*It returns the context ClassLoader for the thread.*
Static Thread.UncaughtExceptionHandler	*getDefaultUncaughtExceptionHandler()*	*It returns the default handler invoked when a thread abruptly terminates due to an uncaught exception.*
static void	*setDefaultUncaughtExceptionHandler()*	*It sets the default handler invoked when a thread abruptly terminates due to an uncaught exception.*

Advantages of Using Multithreading

Here are some of the advantages of using multithreading:

- ***Better CPU Utilization:***
  With multithreading, multiple threads can run concurrently which allows the CPU to execute several tasks at the same time. In this way, CPU resources are fully utilized, reduces the idle time, and improves overall system performance.
- ***Faster Execution:***
  By running tasks in parallel, multithreading can complete operations much faster than sequential execution. For example, multiple calculations or data-processing tasks can be handled simultaneously, that reduces the total execution time.

- ***Improved Responsiveness:***
  In applications with user interfaces, multithreading allows background tasks to run without freezing the main program. This keeps the application responsive to user input, even while performing heavy processing in the background.
- ***Resource Sharing:***
  Threads within the same process share memory and resources that makes communication and data exchange between tasks simpler and more efficient compared to processes that run independently.
- ***Background Processing:***
  Multithreading allows time-consuming tasks to run in the background without interrupting the main workflow.

How to Create a Thread in Java?

Thread is the smallest unit of execution in a Java program that allows multiple tasks to run concurrently.

There are two main ways to create a thread:

By Extending the Thread Class

By Implementing the Runnable Interface

Creating a Thread by Extending Thread Class

When you extend the Thread class, you create a custom thread by overriding its run() method. You then start the thread using the start() method, which moves the thread from the New state to the Runnable state and executes the run() method when the thread gets CPU time.

Example 1: Creating a Thread by Extending Thread Class

The following example demonstrates how to create a custom thread by extending the Thread class and overriding the run() method:

```
class Multi extends Thread {
   public void run() {
      System.out.println("thread is running...");
   }

   public static void main(String args[]) {
      Multi t1 = new Multi();
      t1.start();
   }
}
```

Output:

thread is running...

Here, we define a thread by extending the Thread class. The thread starts executing when we call the start() method.

Example 2: Using Thread(String name) Constructor

The following example demonstrates how to create a thread by directly using the Thread class constructor with a custom name:

```
public class MyThread1 {
   public static void main(String argvs[]) {
      Thread t = new Thread("My first thread");
      t.start();
      System.out.println(t.getName());
   }
}
```

Output:

My first thread

In this example, we create a thread object using the Thread(String name) constructor. The thread is started using the start() method, and we can also retrieve its name using getName().

Example 3: Using Thread(Runnable r, String name) Constructor

The following example demonstrates how to create a thread using the Thread(Runnable r, String name) constructor, which allows specifying both a task and a thread name:

```
public class MyThread2 implements Runnable {
   public void run() {
      System.out.println("Now the thread is running ...");
   }

   public static void main(String argvs[]) {
      Runnable r1 = new MyThread2();
      Thread th1 = new Thread(r1, "My new thread");
      th1.start();
      System.out.println(th1.getName());
   }
}
```

Output:

My new thread

Now the thread is running ...

Here, we create a Runnable task and pass it to the Thread constructor along with a thread name. The thread executes the run() method when started.

Creating a Thread by Implementing the Runnable Interface

Another approach is to implement the Runnable interface. The class must implement the run() method, which contains the task logic. After that, you create a Thread object, passing your Runnable instance to its constructor, and call start().

Example

The following example demonstrates how to create a thread by implementing the Runnable interface:

```
class Multi3 implements Runnable {
   public void run() {
      System.out.println("thread is running...");
   }

   public static void main(String args[]) {
      Multi3 m1 = new Multi3();
      Thread t1 = new Thread(m1);  // Using Thread(Runnable r) constructor
      t1.start();
   }
}
```

Output:

thread is running...

Note: If a class only implements Runnable and does not extend Thread, you must explicitly create a Thread object to execute its run() method.

Thread Scheduler in Java is responsible for deciding the order in which threads are executed by the Java Virtual Machine (JVM). It determines which thread runs at a given time and manages CPU time for multiple threads in a program.

1. Priority

*Each thread has a priority between **1 and 10**. Threads with higher priority are more likely to be chosen by the thread scheduler.*

2. Time of Arrival

*If two or more threads have the **same priority**, the scheduler looks at the **time of arrival**. The thread that became runnable first gets executed first.*

By using these two factors, the thread scheduler manages the execution order of threads and ensures that multiple threads can share CPU time effectively.

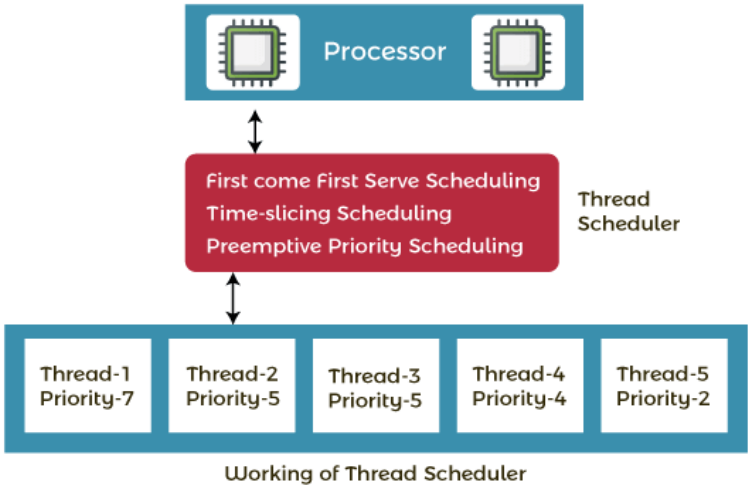
Working of Thread Scheduler with Example

*Let's understand how the thread scheduler works with an example. Suppose there are **five threads** with different **arrival times** and **priorities**. The **thread scheduler** decides which thread gets the CPU first.*

- *The thread with the **highest priority** is selected to run first.*
- *If a thread is already running and a **new thread with higher priority** becomes runnable, the current thread can be **pre-empted**, and the higher-priority thread gets CPU time.*
- *When two threads have the **same priority and arrival time**, the scheduler follows the **First-Come-First-Served (FCFS) principle** that means the thread that arrived first executes first.*

*This ensures that threads are executed efficiently based on **priority** and **arrival time**.*

The following example demonstrates how the thread scheduler selects threads based on priority and arrival time.



Working of Thread Scheduler

Example: Demonstrating Thread Scheduler

The following example demonstrates how the thread scheduler selects threads based on priority and arrival time.

```
class MyThread extends Thread {
   public MyThread(String name, int priority) {
      super(name);         // Set thread name
      setPriority(priority); // Set thread priority (1 to 10)
   }

   public void run() {
      System.out.println(getName() + " with priority " + getPriority() + " is running.");
   }
```

```
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread 1", 3);
        MyThread t2 = new MyThread("Thread 2", 7);
        MyThread t3 = new MyThread("Thread 3", 5);
        MyThread t4 = new MyThread("Thread 4", 7);
        MyThread t5 = new MyThread("Thread 5", 2);

        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t5.start();
    }
     }
```

Output:

```
Thread 2 with priority 7 is running.
Thread 4 with priority 7 is running.
Thread 3 with priority 5 is running.
Thread 1 with priority 3 is running.
Thread 5 with priority 2 is running.
```

Thread Lifecycle

A thread in Java goes through these states:
- **New** → Created but not started (`new Thread()`).
- **Runnable** → Ready to run, waiting for CPU (`start()` called).
- **Running** → Actively executing code.
- **Waiting/Timed Waiting** → Paused, waiting for another thread or a timeout.
- **Terminated** → Finished execution.

⚙️ Runnable Interface
Instead of extending `Thread`, you can implement `Runnable`:
```java
class CashbackUpdater implements Runnable {
    public void run() {
        while(true) {
            System.out.println("Cashback auto-updated!");
            try {
                Thread.sleep(10000); // 10 seconds
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

public class Demo {

public static void main(String[] args) {

Thread t = new Thread(new CashbackUpdater());

t.start();

}

}

```

***Asynchronous Tasks Example***

*Imagine a banking app:*

*- **Main thread** → Handles user input.*

*- **Background thread** → Calculates interest or updates cashback every 10 seconds.*

*This ensures the UI doesn't freeze while backend tasks run.*

---

***Demo: Auto-update Cashback***

*The above code simulates a cashback system:*

*- Every 10 seconds, a background thread updates the balance.*

*- This is asynchronous: the main program continues running while updates happen in parallel.*

*This separates **task logic** (Runnable) from **thread management** (Thread).*

## Parallelism in Backend Operations

Parallelism means multiple tasks run **simultaneously**:

- **Database queries** → Multiple threads fetch data at once.

- **Web servers** → Each client request handled by a separate thread.

- **Financial apps** → Cashback updates, interest calculations, and notifications run in parallel.

This improves **performance** and **responsiveness**. For example:

- One thread updates cashback.

- Another calculates loan interest.

- Another sends notifications.

All without blocking each other.
```